A NOVEL METHODOLOGY OF AI SYSTEM FOR AUTONOMOUS TESTING

A PROJECT REPORT SUBMITTED

IN PARTIAL FULFILMENT OF THE REQUIREMENT

FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

BY

AMAN KUMAR SINGH	**22050445003**
MD AFFAN ALAM	**22050445013**
AMAN KUMAR	**22050445002**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CAMBRIDGE INSTITUTE OF TECHNOLOGY

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

[2026]

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled **AI SYSTEM FOR AUTONOMOUS TESTING** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. RAJNI KANT MUNDA

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA

(Head of Department)

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled **AI SYSTEM FOR AUTONOMOUS TESTING** is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner) **(External Examiner)**

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

We took this opportunity to thank all who have contributed towards shaping this Final Year Project Report. We would like to express our sincere thanks to **Prof. Deepak Kumar Verma** (HOD, Department of Computer Science & Engineering) and to our guide **Prof. Rajni Kant Munda**, whose help in the course of development of this manuscript is invaluable. As our supervisor, he has constantly encouraged us to remain focused on achieving our goal.

We would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. We have been fortunate to have a wonderful support structure among fellow students. Our sincere thanks to everyone who has provided us with kind words, new ideas, useful criticism, or their valuable time. We are truly indebted. Last but not least, we would like to dedicate this work to our family for their love and understanding.

AMAN KUMAR SINGH

22050445003

MD. AFFAN ALAM

22050445013

AMAN KUMAR

22050445002

ABSTRACT

Software testing is an essential activity in the Software Development Life Cycle (SDLC) that ensures application reliability, functionality, and security before deployment. Traditional testing approaches often require extensive manual effort for designing test cases, maintaining automation scripts, executing tests, and analyzing results. As modern applications continue to grow in complexity, conventional testing methods face challenges related to scalability, maintenance overhead, and execution efficiency.

This project, titled " **AI SYSTEM FOR AUTONOMOUS TESTING** ", presents an intelligent platform designed to automate multiple stages of the software testing process. The proposed system integrates repository analysis, artificial intelligence, browser automation, cloud-based execution environments, and centralized reporting mechanisms into a unified workflow. The platform retrieves application source code from GitHub repositories, analyzes project structure, generates relevant testing scenarios using Google Gemini AI, creates executable browser automation scripts, and executes them using Playwright within Browserbase cloud environments.

The system has been developed using Next.js, React, Tailwind CSS, Clerk Authentication, Neon PostgreSQL, Drizzle ORM, Playwright, Browserbase, and Google Gemini AI. These technologies collectively enable secure user management, automated repository processing, intelligent test generation, scalable execution, and comprehensive result tracking.

The implementation demonstrates the feasibility of utilizing Artificial Intelligence to assist software quality assurance activities by reducing manual intervention and accelerating testing workflows. The system provides developers and testers with an efficient mechanism for generating, executing, and reviewing test cases while maintaining visibility through execution logs and session recordings.

The proposed solution contributes toward the development of intelligent software testing platforms capable of improving productivity, enhancing test coverage, and supporting modern cloud-native software development practices.

Keywords: Artificial Intelligence, Test Automation, Google Gemini AI, Repository Analysis, Cloud Browser Execution, Quality Assurance.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

AI Artificial Intelligence

API Application Programming Interface

CI/CD Continuous Integration / Continuous Deployment

CSS Cascading Style Sheets

DBMS Database Management System

DFD Data Flow Diagram

DOM Document Object Model

HTML HyperText Markup Language

HTTP HyperText Transfer Protocol

IDE Integrated Development Environment

JSON JavaScript Object Notation

LLM Large Language Model

QA Quality Assurance

REST Representational State Transfer

SDK Software Development Kit

SQL Structured Query Language

UI User Interface

UML Unified Modeling Language

URL Uniform Resource Locator

UAT User Acceptance Testing

UX User Experience

CHAPTER 1 – INTRODUCTION

1.1 Background and Motivation

Software systems have become an essential component of modern business operations, education platforms, healthcare services, banking systems, e-commerce applications, and cloud-based enterprise solutions. As organizations increasingly rely on digital platforms to deliver services, ensuring software reliability and quality has become a critical requirement. Even minor software defects can lead to financial losses, reduced customer trust, security vulnerabilities, and operational disruptions. Consequently, software testing plays a vital role in validating application functionality before deployment.

Traditionally, software testing has relied on manual verification and predefined automation scripts. Manual testing requires testers to repeatedly perform application workflows and verify expected outcomes. Although effective for exploratory analysis, manual testing becomes inefficient when applications grow in size and complexity. Repetitive testing activities consume significant time and effort, especially in agile development environments where software updates are released frequently.

To overcome these challenges, organizations adopted automated testing frameworks. Tools such as Selenium and Playwright allow developers and testers to automate browser interactions and validate application behavior. While these frameworks improve execution speed and consistency, they still require engineers to manually design, implement, and maintain test scripts. As application interfaces evolve, existing automation scripts often require continuous modification, increasing maintenance costs and reducing overall productivity.

Recent developments in Artificial Intelligence have introduced new opportunities for improving software quality assurance practices. Modern Large Language Models (LLMs) possess the capability to understand source code structures, analyze application workflows, and generate meaningful outputs based on contextual information. These advancements create the possibility of developing intelligent testing platforms capable of assisting engineers throughout the testing lifecycle.

The motivation behind this project originates from the need to reduce the manual effort involved in software testing while improving testing efficiency and scalability. Instead of requiring testers to manually inspect source code and write automation scripts, the proposed system utilizes Artificial Intelligence to analyze application repositories, identify functional areas, generate testing scenarios, and assist in automation script creation. This approach enables developers and testers to focus on application quality and business requirements rather than repetitive scripting tasks.

Furthermore, modern software projects often utilize distributed cloud infrastructures and continuous deployment practices. Testing solutions must therefore support scalable execution environments capable of handling multiple test scenarios efficiently. Cloud-based browser execution platforms provide a practical solution by eliminating local browser dependencies and enabling remote automation workflows. The integration of AI-driven analysis with cloud-based execution forms the foundation of the proposed system.

The project titled ssurancAI SYSTEM FOR AUTONOMOUS TESTING" aims to address these challenges by combining repository analysis, intelligent test generation, browser automation, cloud execution, and centralized reporting within a single platform. By automating key stages of the testing process, the system seeks to improve software quality assurance workflows and support the growing demands of modern software development environments.

1.2 Problem Statement

Software testing is a critical activity performed to verify that an application behaves according to its intended requirements. In contemporary software development environments, applications are frequently updated through Agile and Continuous Integration/Continuous Deployment (CI/CD) practices. As a result, testing teams are required to repeatedly validate existing functionalities while simultaneously verifying newly introduced features.

Most organizations rely on either manual testing procedures or traditional automation frameworks. Manual testing is time-consuming, prone to human error, and difficult to scale for large software projects. On the other hand, conventional automation

frameworks require engineers to manually create, maintain, and update test scripts whenever application interfaces or workflows change. The maintenance effort associated with these scripts often becomes a significant challenge in long-term projects.

Another limitation observed in existing testing approaches is the lack of integration between source code understanding and test generation. Test engineers generally need to analyze application functionality manually before creating suitable test scenarios. This process requires domain knowledge, technical expertise, and considerable development time. Furthermore, maintaining local browser environments, execution infrastructure, and testing dependencies introduces additional operational complexity.

Modern software applications also contain multiple interconnected modules, authentication mechanisms, API integrations, and dynamic user interfaces. Validating these components consistently through manual processes becomes increasingly difficult as application complexity grows. Consequently, organizations face challenges related to delayed releases, incomplete test coverage, higher testing costs, and reduced testing efficiency.

The problem addressed by this project is the absence of an intelligent and integrated platform capable of automatically understanding application repositories, generating meaningful test scenarios, creating executable automation scripts, executing tests in cloud environments, and presenting comprehensive execution results through a centralized dashboard.

The proposed AI SYSTEM FOR AUTONOMOUS TESTING seeks to address these limitations by introducing an AI-assisted testing workflow that combines repository analysis, intelligent test generation, browser automation, cloud execution, and result tracking within a unified platform. The objective is to reduce manual effort while improving testing effectiveness, scalability, and overall software quality.

1.3 Objectives of the Project

The primary objective of this project is to design and develop an intelligent software testing platform capable of assisting developers and quality assurance teams in generating and executing automated test cases with minimal manual intervention.

The specific objectives of the project are listed below:

1. To develop a centralized testing platform capable of managing repository analysis, test generation, execution, and reporting through a unified interface.
2. To integrate GitHub repositories with the platform so that application source code can be analyzed without requiring manual code inspection.
3. To utilize Artificial Intelligence techniques for understanding application structures and generating relevant test scenarios based on repository content.
4. To automate the creation of browser-based testing workflows using Playwright automation technology.
5. To execute generated tests in scalable cloud-based browser environments using Browserbase infrastructure.
6. To capture execution logs, browser sessions, and test outcomes for future analysis and debugging purposes.
7. To provide a structured database system for storing repositories, generated test cases, execution history, and user-related information.
8. To improve software testing productivity by reducing repetitive manual activities associated with test design and script maintenance.
9. To establish a foundation that can support future enhancements such as security validation, self-healing automation, visual testing, and CI/CD pipeline integration.
10. To demonstrate the practical application of Artificial Intelligence in software quality assurance workflows and modern software engineering practices.

1.4 Scope of the Project

The scope of the AI SYSTEM FOR AUTONOMOUS TESTING focuses on the automation of software testing activities for modern web-based applications. The platform is designed to assist developers and testing teams by providing intelligent support for repository analysis, test generation, browser automation, execution monitoring, and result management.

The system primarily targets applications developed using technologies such as React, Next.js, JavaScript, and TypeScript. Through GitHub integration, the platform retrieves repository information and analyzes application structures to generate context-aware testing scenarios. These test cases are transformed into executable automation workflows and executed within cloud-hosted browser environments.

The project includes several major functionalities to support the automated testing workflow. These include user authentication and workspace management, GitHub repository integration and analysis, and Artificial Intelligence-assisted test generation for creating context-aware test scenarios. The platform also provides browser automation using Playwright, cloud-based execution through Browserbase, execution log collection and result storage, dashboard-based monitoring and reporting, and database-driven management of repositories and test cases.

The project is limited to web application testing and does not currently support native mobile applications, desktop software testing, embedded systems, or hardware validation. Advanced features such as visual regression testing, autonomous bug fixing, security vulnerability remediation, and self-healing automation scripts remain outside the scope of the current implementation. Although human review may still be necessary for highly complex business rules or domain-specific workflows, the platform should be considered an intelligent testing assistant rather than a complete replacement for quality assurance professionals. Despite these limitations, it provides a practical framework for integrating Artificial Intelligence into software testing and establishes a strong foundation for future research and development.

1.5 Organization of the Report

This report is organized into six chapters, each addressing a specific aspect of the project development lifecycle.

Chapter 1 introduces the project domain and presents the motivation, problem statement, objectives, scope, and overall purpose of the proposed system. It establishes the need for intelligent software testing solutions within modern software development environments.

Chapter 2 presents a comprehensive review of existing research and industrial practices related to software testing automation, Artificial Intelligence assisted testing, browser automation frameworks, cloud-based execution platforms, and autonomous quality assurance systems. A comparative analysis is also performed to identify research gaps and justify the proposed solution.

Chapter 3 discusses the analysis and design of the system. This chapter presents hardware and software requirements, architectural design, data flow diagrams, entity relationship diagrams, and UML models that describe the structure and interaction of system components.

Chapter 4 focuses on the implementation details of the project. It explains the technologies used, module-wise functionality, workflow execution, repository analysis process, AI-driven test generation mechanism, browser automation techniques, database design, and reporting capabilities. Relevant implementation screenshots and code excerpts are included to support the discussion.

Chapter 5 describes the testing methodology used to evaluate the system. Test cases, execution results, performance observations, and system behavior under various operational scenarios are analyzed and presented.

Chapter 6 concludes the report by summarizing the achievements of the project, evaluating its contributions, discussing identified limitations, and outlining future enhancements that can improve the capabilities of the platform.

The report concludes with references, source code appendices, and supplementary screenshots that provide additional technical details regarding the implementation and operation of the system.

CHAPTER 2 – LITERATURE REVIEW

2.1 Related Works

Bertolino (2007) presented a comprehensive, foundational analysis mapping the historical milestones, major theoretical achievements, and future paradigms within the domain of software testing research. The primary objective of this study was to outline a long-term research roadmap transitioning from human-intensive manual validation routines toward autonomous, continuous, and self-adaptive quality assurance ecosystems. By evaluating the systemic evolution of test automation from simple linear scripts to complex data-driven structures, the author contributed a rigorous conceptual framework that highlighted the ultimate necessity of embedding computational intelligence directly into the software lifecycle to keep pace with rapid agile development iterations. The findings emphasized that human cognitive constraints and script authoring overhead would eventually cause critical bottlenecks in high-velocity deployment pipelines unless test suites became natively context-aware. However, because this landmark work was compiled during an early computing era, it remains primarily focused on abstract theoretical methodologies and lacks the architectural capacity to address modern cloud browser orchestration, isolated microservice containers, or generative language models.

Harman et al. (2015) investigated the primary advancements, open computational complexities, and structural engineering hurdles associated with Search Based Software Testing (SBST) frameworks. The objective of this study was to design and evaluate optimization algorithms capable of mathematically traversing vast input execution spaces to autonomously discover edge cases and maximize branch coverage metrics. Utilizing meta-heuristic search techniques such as genetic algorithms and simulated annealing, the researchers contributed to the field by automating test data generation and test path generation without relying on tedious manual script authoring. The experimental results demonstrated that SBST methodologies can efficiently pinpoint hidden application faults and radically optimize regression test suites. Nevertheless, a notable limitation identified within this research is its absolute dependency on precisely formulated fitness functions and deterministic constraints. The approach lacks high-level semantic context awareness, rendering it fundamentally incapable of reading

codebase architecture, parsing multi-factor authentication workflows, or adaptively responding to sudden mutations in dynamic application user interfaces.

Felderer and Hasselbring (2021) presented a detailed overview of the core challenges, industrial shifts, and evolutionary trajectories of software quality assurance in the emerging era of machine intelligence. The primary objective of this research was to establish a formal taxonomy classifying testing approaches based on their reliance on artificial intelligence for test suite generation, programmatic browser execution, and semantic defect analysis. The study contributed a robust theoretical framework emphasizing the critical necessity of seamlessly embedding data-driven validation components throughout continuous integration loops. Their results indicated that traditional static testing scripts fall short when validating highly complex, cloud-native web applications, highlighting a clear pathway for intelligent automation systems. Although this work provides an exceptional conceptual foundation for modern software quality assurance, it remains high-level. The authors focused predominantly on macro industry classifications and did not address specific execution sandboxes, concrete prompt engineering rules, or multi-agent synchronization strategies.

S. E. et al. (2021) conducted an extensive systematic mapping study evaluating the scalability parameters, infrastructure configurations, and multi-tenant performance of cloud-based software testing platforms. The research objective was to analyze how distributed cloud architectures minimize localized hardware dependencies and optimize computational resource allocation during massive browser testing cycles. By organizing empirical data across numerous distributed systems, the authors contributed a definitive architectural paradigm showing that remote browser parallelization minimizes release lifecycle blockages. The key findings verified that running tests across isolated cloud containers optimizes execution velocity and eliminates hardware bottlenecks. However, the investigation identified a persistent limitation regarding the inherent latency constraints introduced by container cold-starts and network data transfers. Furthermore, the framework relied entirely on pre-authored, static test scripts, completely lacking any automated mechanisms to independently discover application routing paths or synthesize contextual test parameters.

Stocco et al. (2021) investigated the long-term technical debt, maintenance challenges, and systemic operational failures associated with user interface automation scripts

within dynamic web systems. The objective of this longitudinal study was to analyze the root causes, visual symptoms, and underlying programmatic mechanics of test breakage within dynamic, component-driven web applications. Through a rigorous empirical evaluation of Document Object Model (DOM) mutations, the researchers contributed a vital verification showing that traditional, static locator strategies—such as strict XPath or CSS bindings—are excessively brittle. Minor frontend layout alterations inevitably lead to catastrophic script crashes, forcing engineering teams to invest massive human effort into manual script updates. The findings established a critical research gap and a foundational need for adaptive, context-aware web testing architectures. Their work underscores that software engineering requires testing platforms capable of dynamically interpreting element semantics and self-healing layout locators without human intervention.

Dong et al. (2022) proposed an automated self-healing test framework for web applications designed to resolve the locator fragility highlighted in prior literature. The objective of this study was to combine machine learning algorithms with computer vision models to dynamically recognize interface element mutations and patch broken test scripts at runtime. The authors contributed a proactive element recovery methodology that successfully intercepts locator execution errors, evaluates the altered layout structure, and maps old elements to new DOM nodes. Their results demonstrated a substantial improvement in continuous integration pipeline stability and a measurable reduction in manual maintenance overhead. Nevertheless, a critical limitation of this architecture is its purely reactive nature. While the system excels at repairing pre-existing broken locators, it cannot autonomously analyze code repositories, extract application routing logic, or synthesize fresh, comprehensive end-to-end testing scenarios from scratch.

Pearce et al. (2022) systematically evaluated the security vulnerabilities, code injection vectors, and structural flaws inherent in source code synthesized by generative artificial intelligence models. The objective of this study was to assess the security hygiene of automated code generation assistants across multi-tenant software environments. Utilizing empirical boundary evaluations, the authors contributed a critical security warning by demonstrating that language models frequently generate code blocks containing severe, well-documented security vulnerabilities. Their findings emphasize that executing non-deterministic, machine-generated code locally introduces critical

Remote Code Execution (RCE) risks that could compromise host operating systems. This study establishes a foundational architectural constraint for the software testing field; any platform utilizing automated script synthesis must execute those scripts within strictly isolated, ephemeral cloud sandbox containers. However, the paper focused exclusively on passive code generation analysis and did not provide execution sandboxing frameworks for real-time web testing workflows.

Castro et al. (2023) undertook a comprehensive empirical evaluation comparing modern web user interface testing frameworks, specifically evaluating the operational characteristics of Playwright, Cypress, and Selenium. The research objective was to analyze execution speeds, framework stability, synchronization capabilities, and CPU overhead across standardized enterprise application testing scenarios. The authors contributed a definitive performance mapping confirming that Playwright delivers exceptional execution velocity and superior synchronization due to its native bidirectional connection via the Chrome DevTools Protocol (CDP). The findings demonstrated that modern asynchronous web architectures are validated with drastically reduced false-positive failure rates under Playwright. Despite these critical insights, the study highlighted a fundamental limitation shared across all three frameworks. They remain entirely dependent on static, manually compiled script configurations, lacking any embedded machine intelligence to autonomously interpret underlying repository context, generate assertions, or construct dynamic browser workflows without human script creation.

Lemieux et al. (2023) introduced CODAMOSA, an advanced hybrid software validation framework designed to transcend traditional code coverage limitations. The objective of this research was to pair pre-trained Large Language Models with Search Based Software Testing (SBST) to escape optimization plateaus. The methodology utilized the LLM as a context-aware mutation engine; when the deterministic algorithm plateaus, the system prompts the AI to generate test inputs targeting deeply nested, hard-to-reach code blocks. The authors contributed a significant advancement in automated test case generation, demonstrating an increase in overall branch coverage across complex software targets. The key findings illustrated that blending semantic model insights with rigid algorithmic bounds minimizes hallucinations while maximizing verification accuracy. However, a key research gap remains: the framework was designed primarily for localized unit-level fuzzing and did not incorporate global

repository architecture parsing, cloud-native browser orchestration, or secure web authentication validation.

Zhang et al. (2023) developed RepoCoder, an innovative repository-level code completion and context processing framework built to enhance artificial intelligence code understanding. The primary research objective was to bypass traditional file-size limits by introducing an iterative retrieval-generation loop capable of contextually slicing expansive software structures. By engineering an algorithm that searches for cross-file structural dependencies and isolates vital code snippets, the authors contributed a method for packaging highly dense, semantic data payloads for Large Language Models. Their results proved that repository-aware context slicing drastically improves the syntactic precision of AI outputs compared to standard monolithic prompts. Nevertheless, a notable limitation of this study is its singular focus on static code generation. The framework lacked a functional execution pipeline or a closed-loop validation engine, meaning it could not translate its repository comprehension into runnable browser automation files or capture runtime telemetry.

Schäfer et al. (2024) conducted a large-scale, empirical evaluation analyzing the precision, structural validity, and logical limits of Large Language Models tasked with automated test generation. The objective of this study was to map the capabilities of modern generative models to write executable, non-trivial test suites directly from program parameters. The methodology focused on testing various prompt engineering structures against multiple object-oriented language targets. The researchers contributed a critical architectural guideline, demonstrating that while LLMs successfully identify semantic testing goals, their output is highly unstable and prone to formatting errors without rigid boundary constraints. The findings proved that schema-based response validation is absolutely mandatory to prevent hallucinated logic from disrupting automated testing pipelines. A distinct limitation identified in their work is that the generation process remained disconnected from cloud-native execution environments, requiring manual human verification to compile and run the synthesized testing files locally.

Wang et al. (2024) presented an extensive, state-of-the-art survey detailing the landscape, cognitive architectures, and future milestones of software testing driven by Large Language Models. The objective of this investigation was to evaluate the

paradigm shift from single-prompt systems toward multi-agent networks specializing in discrete software engineering phases. The authors contributed a foundational vision of autonomous validation ecosystems, detailing how decoupling cognitive responsibilities among specialized agents (planners, coders, and execution monitors) exponentially increases logical precision and bypasses context window limits. Their findings demonstrated that collaborative multi-agent orchestration minimizes systemic bugs and provides a scalable path for autonomous quality assurance. However, the study concluded that practical end-to-end integrations remain an open research challenge. Most existing tools operate as disjointed prototypes that lack unified persistence layers, token-billing management, secure multi-factor authentication handling, or isolated cloud-sandbox execution spaces to protect host platforms.

The progressive trajectory of the reviewed literature clearly establishes an evolutionary paradigm shift from manual scripting toward isolated, context-aware machine intelligence. However, a systemic research gap persists across existing systems: test generation engines, cloud-native browser infrastructures, version-control parsers, and security verification modules continue to exist as isolated academic or commercial implementations. This lack of architectural convergence forces engineering teams to manage fragmented tools, resulting in continuous manual overhead and high maintenance technical debt. The proposed project, **AI SYSTEM FOR AUTONOMOUS TESTING**, directly addresses these foundational gaps by engineering a unified, closed-loop multi-agent ecosystem. By seamlessly consolidating real-time repository contextual filtering, schema-bounded generative AI test case synthesis, secure ephemeral browser execution via Browserbase, and multi-factor telemetry logging, the platform introduces an entirely integrated, self-adaptive solution that transitions software quality assurance from static test automation into absolute execution autonomy.

Ref. No.	Author(s) & Year	Title of the Work	Methodology/ Approach	Key Contribution	Limitation
[1]	Bertolino (2007)	Software Testing Research: Achievements, Challenges, Dreams	Evolutionary mapping and theoretical roadmapping of testing paradigms.	Outlined a comprehensive framework transitioning manual validation to autonomous software assurance.	Focuses on abstract theoretical methodologies; completely lacks cloud or AI execution capabilities.

Ref. No.	Author(s) & Year	Title of the Work	Methodology/ Approach	Key Contribution	Limitation
[2]	Harman et al. (2015)	Achievements, Open Problems and Challenges for Search Based Software Testing	Meta-heuristic search algorithms (genetic algorithms and simulated annealing).	Automates test data synthesis and execution path generation without manual script authoring.	Dependent on precisely formulated fitness functions; lacks high-level semantic context awareness.
[3]	Felderer & Hasselbring (2021)	Software Testing in the Age of Artificial Intelligence	Analytical classification and formal taxonomy of AI quality assurance.	Categorized software testing methods based on their specific reliance on AI for script generation and defect analysis.	High-level macro industry framework; does not address specific prompt engineering or multi-agent rules.
[4]	S. E. et al. (2021)	Cloud-Based Software Testing: A Systematic Mapping Study	Empirical mapping and cross-evaluation of distributed cloud test setups.	Demonstrated that remote browser parallelization optimizes execution speed and drops hardware constraints.	Vulnerable to cloud container cold-starts; relies entirely on pre-authored, static test scripts.
[5]	Stocco et al. (2021)	Visual Web Test Breakage: Causes, Symptoms, and Remedies	Longitudinal empirical tracking of DOM mutations and script breakdown.	Verified that traditional locator strategies (XPath/CSS bindings) accumulate massive script maintenance debt.	Diagnoses and quantifies test suite failure modes but provides no automated remediation system.
[6]	Dong et al. (2022)	Self-Healing Test Scripts for Web Applications	Machine learning models paired with computer vision layout analysis.	Created a proactive runtime element recovery mechanism that dynamically self-heals broken locators.	Purely reactive execution tier; cannot autonomously scan code repositories to generate new test scripts.
[7]	Pearce et al. (2022)	Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions	Empirical boundary assessment and safety hygiene analysis of AI synthesis.	Highlighted critical security vulnerabilities in AI code, establishing the absolute necessity for execution sandboxes.	Passive analysis of vulnerabilities; does not implement an operational sandbox for cloud testing workflows.
[8]	Castro et al. (2023)	A Comprehensive Empirical Study on Web Testing with Playwright, Cypress, and Selenium	Cross-framework empirical performance evaluation across web systems.	Proved Playwright delivers superior execution speed and synchronization via native DevTools protocol bindings.	Test scripts remain completely static and dependent on continuous manual developer authorship.

Ref. No.	Author(s) & Year	Title of the Work	Methodology/ Approach	Key Contribution	Limitation
[9]	Lemieux et al. (2023)	CODAMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models	Hybrid execution pairing pre-trained LLMs with search-based testing.	Utilized language models as context-aware mutation engines to bypass code coverage plateaus.	Confined to unit-level generation; lacks support for global repository analysis or cloud browser interfaces.
[10]	Zhang et al. (2023)	RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation	Iterative retrieval-generation loop optimization framework.	Introduced recursive repository context slicing to drastically maximize the syntactic accuracy of LLM outputs.	Focused solely on static code generation; lacks runtime execution sandboxes or a closed-loop telemetry system.
[11]	Schäfer et al. (2024)	An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation	Systematic prompt engineering analysis against object-oriented targets.	Proved that schema-bounded response formats are mandatory to reliably eliminate AI generation hallucinations.	Fully isolated from the execution runtime; requires manual compilation and localized execution verification.
[12]	Wang et al. (2024)	Software Testing with Large Language Models: Survey, Landscape, and Vision	State-of-the-art literature survey of LLM validation structures.	Outlined how decoupling cognitive steps among specialized multi-agent systems optimizes logical precision.	Conceptual synthesis; does not build a unified ecosystem with cloud sandboxing or database persistence.

Table 2.1 – Summary of Related Works

2.2 Comparative Analysis

The literature survey demonstrates substantial progress in software testing automation, Artificial Intelligence-assisted testing, repository analysis, cloud-based execution environments, and autonomous software quality assurance systems. However, a detailed examination of existing approaches reveals that most solutions focus on specific stages of the testing lifecycle rather than providing a complete end-to-end testing ecosystem.

Traditional automation frameworks such as Selenium and Playwright have significantly improved the efficiency of browser-based testing. These tools enable automated execution of repetitive testing activities and reduce human effort during validation

processes. Nevertheless, they depend heavily on manually developed test scripts. Any modification to application workflows or user interfaces frequently requires script maintenance, leading to increased operational costs and reduced testing agility.

Research involving Artificial Intelligence and Large Language Models has demonstrated promising results in automated test case generation. Studies indicate that AI systems can analyze source code, application requirements, and functional descriptions to produce meaningful testing scenarios. While these approaches reduce the effort associated with test design, many of them stop at the generation phase and do not provide integrated execution, monitoring, or result management functionalities.

Repository analysis techniques have further improved the ability of intelligent systems to understand software applications. By extracting contextual information directly from source code repositories, AI models can generate more relevant and application-specific testing artifacts. However, repository analysis alone is insufficient without execution infrastructure capable of validating the generated test cases.

Cloud-based browser automation platforms have addressed infrastructure challenges by enabling scalable execution of browser tests without requiring local environment management. Such platforms simplify deployment and improve resource utilization. Despite these benefits, they generally rely on externally authored test scripts and do not provide intelligent mechanisms for generating or adapting testing workflows.

A comparison of the surveyed works indicates that existing solutions typically address one or two aspects of software testing, such as automation, AI-based generation, repository analysis, or execution infrastructure. Very few systems combine these capabilities within a unified framework.

The proposed AI SYSTEM FOR AUTONOMOUS TESTING attempts to bridge this gap by integrating repository analysis, Artificial Intelligence-driven test generation, Playwright-based automation, Browserbase cloud execution, centralized database management, and reporting capabilities into a single platform. Unlike traditional frameworks that require extensive manual scripting, the proposed system utilizes repository-aware AI processing to assist in generating testing scenarios. Furthermore, the generated tests can be executed within cloud-hosted browser environments while maintaining centralized storage of execution logs, results, and repository information.

Feature	Traditional Manual Testing	Selenium-Based Automation	AI Test Generation Systems	Cloud Browser Platforms	Proposed AI System for Autonomous Testing
Manual Test Creation	High	High	Low	High	Low
Repository Understanding	No	No	Partial	No	Yes
AI-Assisted Test Generation	No	No	Yes	No	Yes
Automated Script Creation	No	Partial	Yes	No	Yes
Cloud-Based Execution	No	No	No	Yes	Yes
Execution Monitoring	Manual	Limited	Limited	Yes	Yes
Centralized Reporting	Limited	Limited	Partial	Partial	Yes
Scalability	Low	Medium	Medium	High	High
Maintenance Effort	High	High	Medium	Medium	Low
End-to-End Integration	No	No	Partial	Partial	Yes

Table 2.2 – Comparative Analysis of Existing Approaches and Proposed System

CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN

3.1 System Requirements

3.1.1 Hardware Requirements

The AI SYSTEM FOR AUTONOMOUS TESTING has been developed as a modern web-based platform that utilizes Artificial Intelligence, cloud-based browser automation, and centralized database management. Since the application is primarily deployed through web technologies and cloud services, the hardware requirements remain moderate for development and operational use.

The development environment requires a system capable of running modern web frameworks, database clients, browser automation tools, and integrated development environments simultaneously. Adequate processing power and memory are necessary to ensure smooth execution of local development activities, repository analysis, and testing workflows.

Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i3 (8th Gen) / Equivalent	Intel Core i5/i7 (10th Gen or Above)
RAM	8 GB	16 GB or Higher
Storage	256 GB SSD	512 GB SSD or Higher
Internet Connection	Broadband (10 Mbps)	High-Speed Broadband (50 Mbps+)
Display Resolution	1366 × 768	1920 × 1080
Browser Support	Chrome, Edge, Firefox	Latest Chrome or Edge
Network	Stable Internet Access	High-Speed Continuous Connectivity

Table 3.1 – Hardware Requirements

3.1.2 Software Requirements

The proposed system integrates several modern technologies to provide a scalable and intelligent testing platform. These technologies collectively support repository management, Artificial Intelligence integration, browser automation, cloud execution, authentication, and database operations.

The frontend layer is developed using Next.js and React, enabling responsive user interfaces and efficient client-server communication. The backend functionality is implemented using Next.js API routes and Node.js runtime services. Google Gemini AI is utilized for intelligent test generation, while Browserbase and Playwright provide browser automation capabilities. Neon PostgreSQL serves as the primary database platform, and Drizzle ORM simplifies database interactions.

Software Component	Purpose
Windows 10/11 or Linux	Operating System
Visual Studio Code	Development Environment
Node.js	Backend Runtime Environment
Next.js	Full-Stack Web Framework
React.js	Frontend Development
Tailwind CSS	User Interface Styling
Shadcn UI	UI Component Library
Google Gemini AI	Intelligent Test Generation
GitHub API	Repository Access and Analysis
Playwright	Browser Automation Framework

Software Component	Purpose
Browserbase	Cloud Browser Execution
Neon PostgreSQL	Database Management System
Drizzle ORM	Database Interaction Layer
Clerk	Authentication and User Management
Stripe	Subscription and Credit Management
Git	Version Control System
Postman	API Testing and Validation

Table 3.2 – Software Requirements

3.1.3 Feasibility Analysis

Feasibility analysis is performed to determine whether the proposed system can be successfully developed, deployed, and maintained within available resources and constraints.

Technical Feasibility

The proposed **AI SYSTEM FOR AUTONOMOUS TESTING** utilizes widely adopted technologies such as Next.js, React, Clerk Authentication, Google Gemini AI, Playwright, Browserbase, Neon PostgreSQL, and Drizzle ORM. These technologies are readily available, well-documented, and supported by active developer communities. Since the required technical resources and development expertise are available, the project is considered technically feasible.

Economic Feasibility

The system primarily relies on open-source technologies and cloud-based services with free-tier support during development. Infrastructure requirements are minimal because browser execution is performed using Browserbase cloud environments and database services are hosted using serverless platforms.

The development cost is significantly lower compared to traditional enterprise testing infrastructures. Therefore, the project is economically feasible for academic and small-scale organizational use.

Operational Feasibility

The proposed system is designed with a user-friendly interface that simplifies repository management, test generation, execution monitoring, and result analysis. Users are not required to possess advanced automation testing expertise to utilize the platform effectively.

The integration of AI-assisted workflows further reduces operational complexity and improves usability. Consequently, the system is considered operationally feasible for software developers, testers, and quality assurance teams.

3.1.4 Existing System vs Proposed System

A comparison between traditional software testing approaches and the proposed AI SYSTEM FOR AUTONOMOUS TESTING is presented in Table 3.3.

Feature	Existing Systems	Proposed System
Test Case Creation	Manual	AI-Assisted
Script Development	Manual Coding	Automated Generation
Repository Understanding	Not Available	Repository-Aware Analysis
Browser Execution	Local Environment	Cloud-Based Browserbase
Reporting	Limited	Centralized Dashboard
Scalability	Moderate	High
Test Maintenance	High Effort	Reduced Effort
AI Integration	Not Available	Integrated Gemini AI
Execution Monitoring	Limited	Detailed Logs & Recordings
User Productivity	Moderate	Improved

Table 3.3 – Existing System vs Proposed System

The comparison demonstrates that the proposed system integrates Artificial Intelligence, repository analysis, cloud execution, and centralized reporting into a unified testing environment, reducing manual effort and improving testing efficiency.

3.2 System Architecture

The architecture of the AI SYSTEM FOR AUTONOMOUS TESTING follows a modular and service-oriented design. The system combines repository analysis, Artificial Intelligence processing, browser automation, cloud execution, and result management into a unified workflow.

The architecture consists of six primary layers:

1. User Interaction Layer
2. Authentication Layer
3. Repository Intelligence Layer
4. AI Processing Layer
5. Test Execution Layer
6. Data Management and Reporting Layer

The workflow begins when a user authenticates through the Clerk authentication service and accesses the dashboard. After successful authentication, the user connects a GitHub repository through OAuth integration. Repository information is retrieved and analyzed to identify relevant project structures and testing opportunities.

The repository context is then supplied to the Artificial Intelligence processing layer, where Google Gemini AI generates testing scenarios and structured test specifications. These generated artifacts are subsequently converted into executable automation workflows using Playwright.

The execution layer utilizes Browserbase cloud browsers to perform automated interactions with the target application. During execution, screenshots, logs, session recordings, and test outcomes are collected and transmitted to the reporting subsystem.

Finally, execution results are stored within the Neon PostgreSQL database through Drizzle ORM. The reporting layer retrieves this information and presents it through interactive dashboards and analytics interfaces.

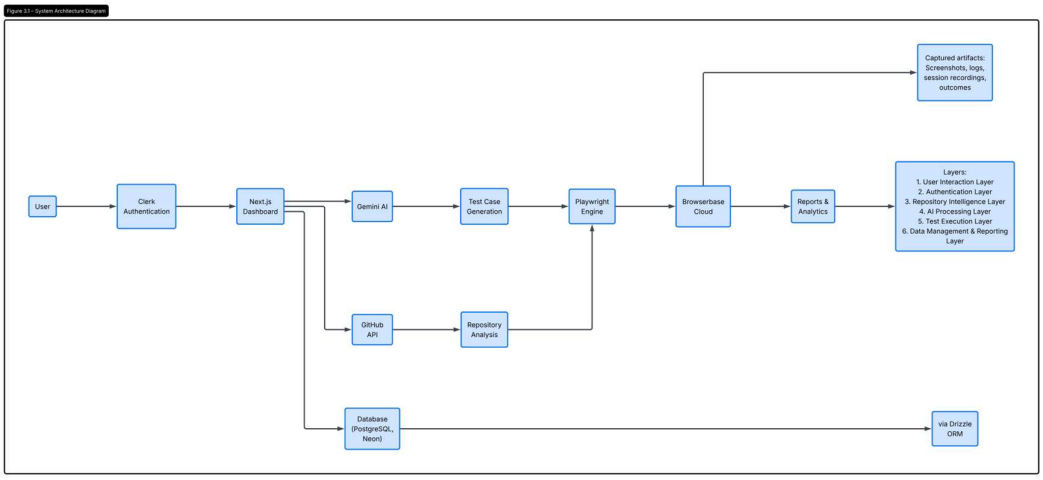


Figure 3.1 – System Architecture Diagram

The architecture provides a clear separation of responsibilities while maintaining efficient communication between modules. This modular design simplifies maintenance, improves scalability, and supports future enhancements such as security validation, CI/CD integration, and advanced analytics capabilities.

3.3 Data Flow Diagrams

3.3.1 Data Flow Diagram (Level 0)

A Data Flow Diagram (DFD) is used to represent the movement of information within a system. The Level 0 DFD, also known as the Context Diagram, provides a high-level view of the AI SYSTEM FOR AUTONOMOUS TESTING and illustrates its interaction with external entities.

In the proposed system, the primary external entity is the user, who interacts with the platform through a web-based dashboard. The user submits repository information, requests test generation, initiates execution processes, and reviews generated reports. The system communicates with external services including GitHub for repository retrieval, Google Gemini AI for intelligent test generation, and Browserbase for cloud-based browser execution.

The system processes incoming requests, generates test artifacts, executes automated workflows, stores execution information, and provides analytical reports through a centralized interface.

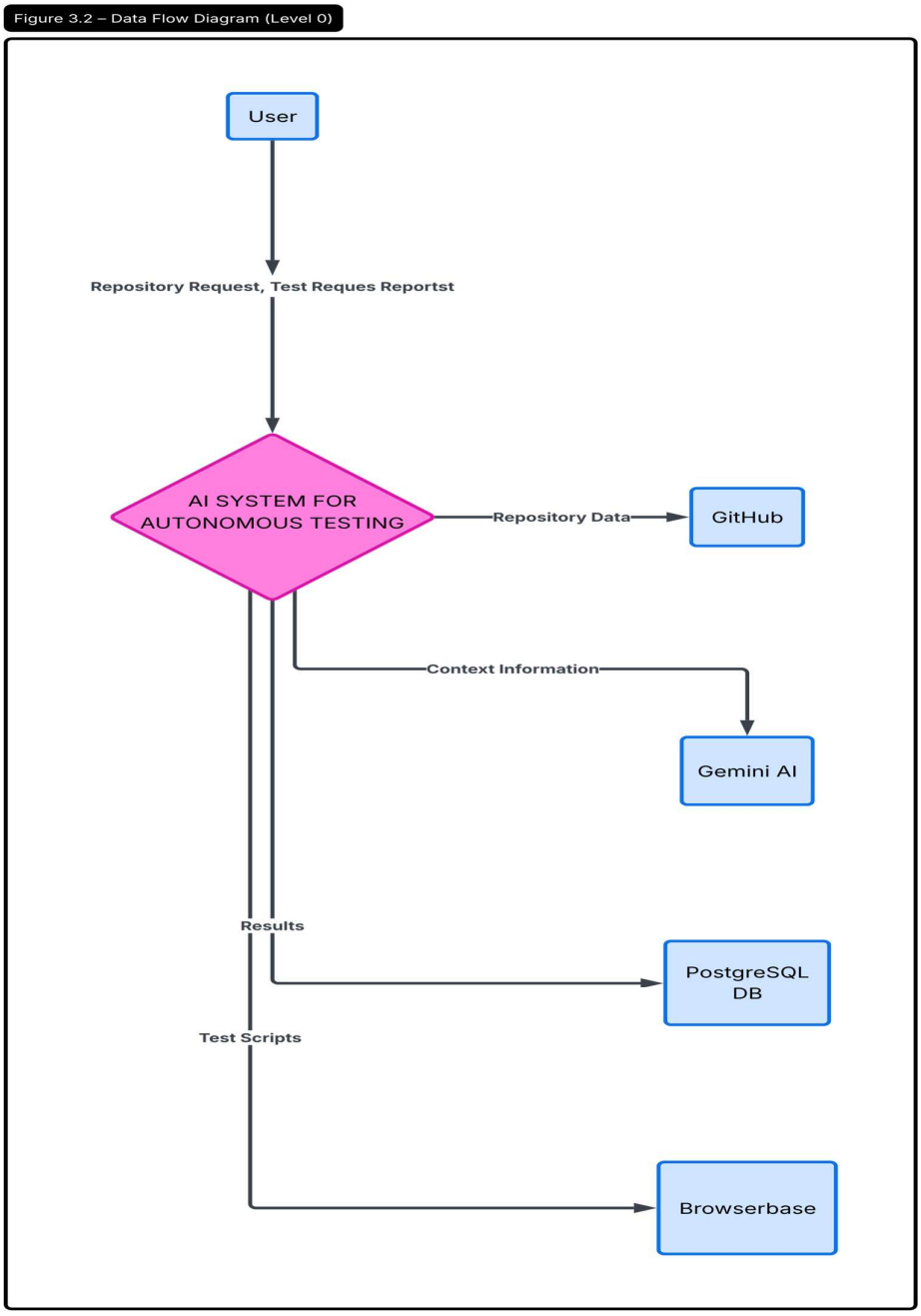


Figure 3.2 – Data Flow Diagram (Level 0)

The Level 0 DFD demonstrates that the system acts as a centralized intelligent testing platform connecting repository sources, AI services, cloud execution environments, and end users through a unified workflow.

3.3.2 Data Flow Diagram (Level 1)

The Level 1 Data Flow Diagram expands the internal processes represented in the Level 0 diagram. It illustrates how information moves between individual modules within the AI SYSTEM FOR AUTONOMOUS TESTING.

The workflow begins when a user authenticates through the authentication module and accesses the dashboard. Once authenticated, the repository analysis module retrieves repository information from GitHub and extracts contextual data required for test generation.

The extracted repository context is forwarded to the AI processing module, where Google Gemini AI analyzes project structures and generates testing scenarios. These scenarios are transformed into executable automation workflows through the Playwright execution engine.

The execution module submits generated scripts to Browserbase cloud browsers for automated testing. During execution, logs, screenshots, session recordings, and test outcomes are captured and stored within the database subsystem. Finally, the reporting module retrieves stored information and generates dashboards and execution reports for end users.

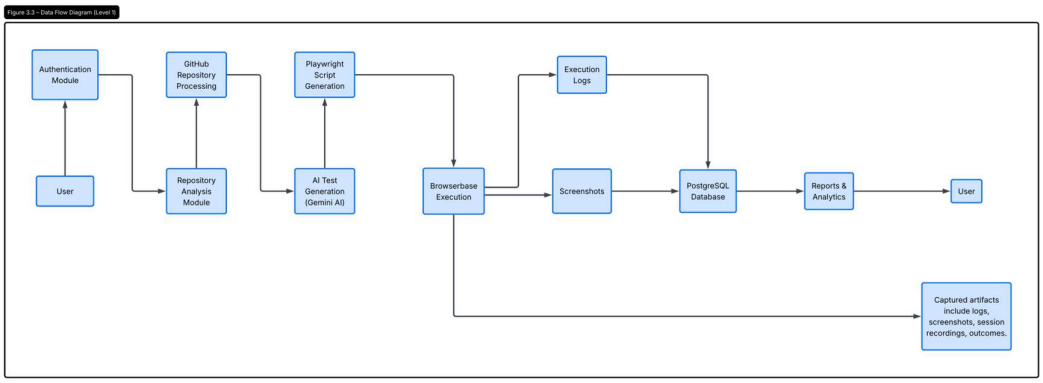


Figure 3.3 – Data Flow Diagram (Level 1)

The Level 1 DFD provides a more detailed representation of the internal processes responsible for repository analysis, AI-assisted testing, browser automation, execution monitoring, and reporting.

3.4 UML Diagrams

3.4.1 UML Class Diagram (Database Entity Model)

Introduction

The UML Class Diagram represents the logical data structure of the **AI SYSTEM FOR AUTONOMOUS TESTING**, illustrating the primary classes, their attributes, and relationships. It provides a structured view of repository management, AI-generated testing, cloud-based execution, reporting, and billing functionalities within the platform. Unlike workflow diagrams that describe system behaviour, the UML Class Diagram focuses on the static structure of the application and serves as a logical representation of the database model implemented within the system.

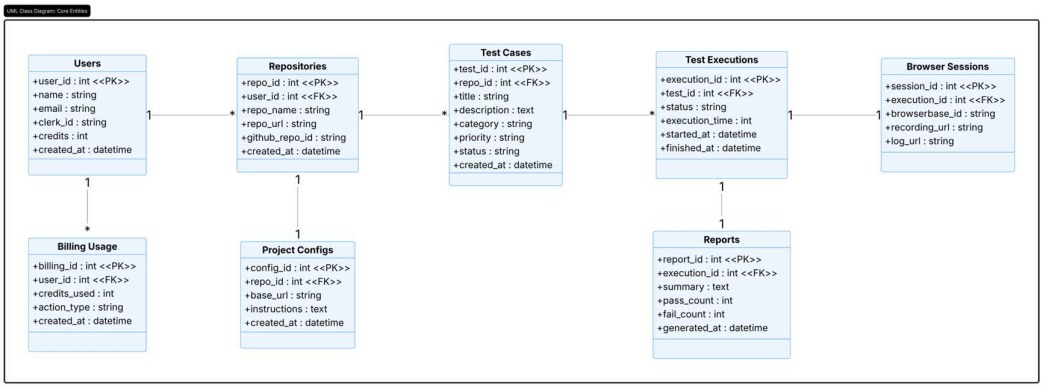


Figure 3.4 – UML Class Diagram (Database Entity Model)

Description of Classes

1. Users Class

The **Users** class represents authenticated users of the platform. It stores essential user information including identity details, authentication references, available testing credits, and account creation timestamps.

The Users class serves as the parent entity for repositories and billing records.

2. Repositories Class

The **Repositories** class stores GitHub repositories connected by users. It maintains repository metadata and links repositories to their respective owners.

Repositories act as the central source of information for AI analysis and test generation.

3. Project Configs Class

The **Project Configs** class stores project-specific testing configurations.

These configurations guide AI processing and test generation activities.

4. Test Cases Class

The **Test Cases** class stores software testing scenarios generated by the AI engine after analyzing repository contents.

This class represents the core output produced by the AI testing engine.

5. Test Executions Class

The **Test Executions** class stores information related to the execution of generated test cases.

Multiple executions may be performed for the same test case during different testing cycles.

6. Browser Sessions Class

The **Browser Sessions** class stores Browserbase cloud browser session information generated during test execution.

This class supports debugging, monitoring, and execution traceability.

7. Reports Class

The **Reports** class stores summarized execution results and testing statistics.

Reports provide consolidated testing insights to users through dashboards and analytics interfaces.

8. Billing Usage Class

The **Billing Usage** class tracks platform credit consumption and billing activities.

This class supports subscription management and usage monitoring.

Relationship Analysis

The relationships among the classes define how information flows throughout the platform.

Users → Repositories

A single user can connect multiple GitHub repositories.

Relationship: One-to-Many (1:M)

User (1) ---------- (*) Repository

Users → Billing Usage

A user may generate multiple billing and credit usage records over time.

Relationship: One-to-Many (1:M)

User (1) ---------- (*) Billing Usage

Repositories → Project Configs

Each repository maintains a project configuration containing testing preferences and target application settings.

Relationship: One-to-One (1:1)

Repository (1) ---------- (1) Project Config

Repositories → Test Cases

AI analysis of a repository can generate multiple test cases.

Relationship: One-to-Many (1:M)

Repository (1) ---------- (*) Test Case

Test Cases → Test Executions

A test case may be executed multiple times as part of repeated testing cycles.

Relationship: One-to-Many (1:M)

Test Case (1) ---------- (*) Test Execution

Test Executions → Browser Sessions

Each execution creates a Browserbase cloud browser session.

Relationship: One-to-One (1:1)

Test Execution (1) ---------- (1) Browser Session

Test Executions → Reports

Each execution generates a corresponding report summarizing testing outcomes.

Relationship: One-to-One (1:1)

Test Execution (1) ---------- (1) Report

Overall Analysis

The UML Class Diagram demonstrates the logical organization of the AI SYSTEM FOR AUTONOMOUS TESTING. The model establishes a clear relationship between authenticated users, connected repositories, AI-generated test cases, execution records, Browserbase sessions, reporting mechanisms, and billing information. This structure enables efficient repository analysis, automated test generation, cloud-based execution, centralized reporting, and resource management within a unified testing platform.

The diagram serves as the foundational data model supporting the implementation of the proposed system and ensures consistency across the authentication, repository management, artificial intelligence, execution, reporting, and billing modules.

3.4.2 Use Case Diagram

The Use Case Diagram identifies the interactions between system users and the functional modules of the platform. The primary actor is the authenticated user who performs repository management, test generation, execution monitoring, and report analysis activities.

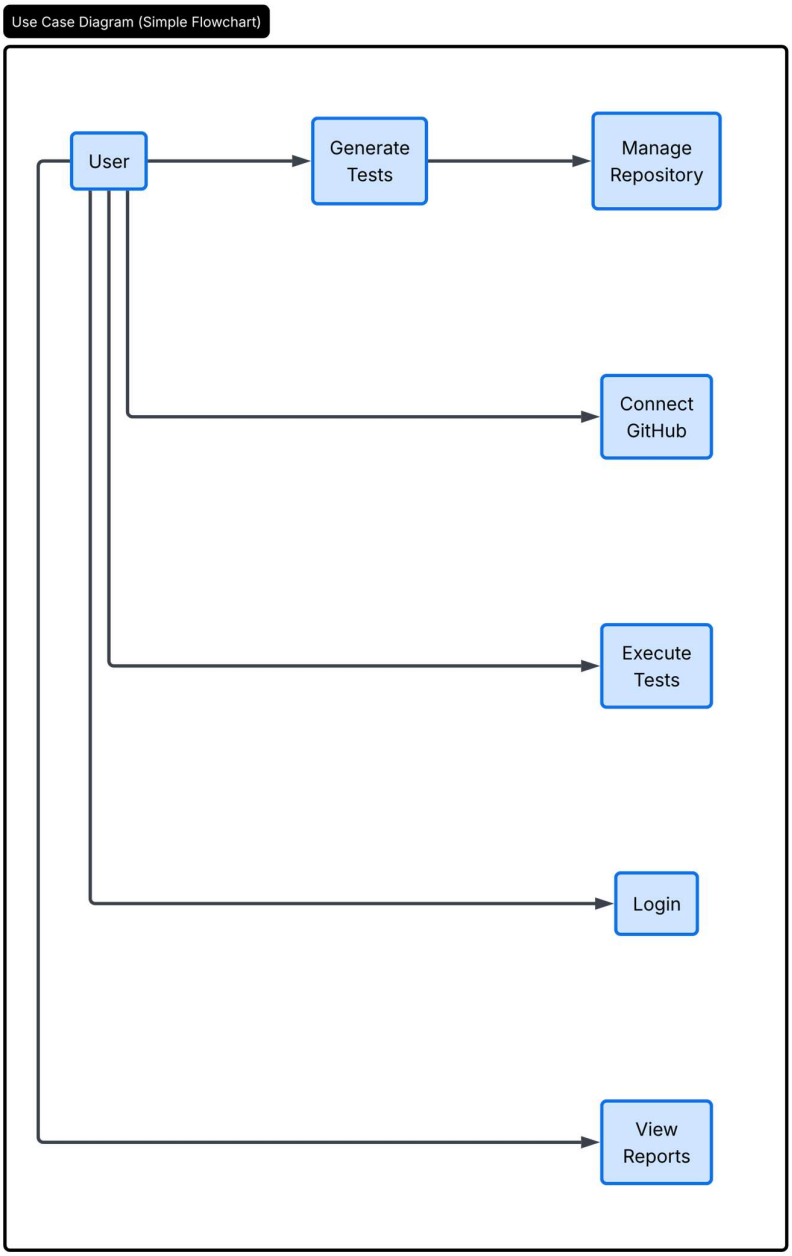


Figure 3.5 – Use Case Diagram

3.4.3 Sequence Diagram

The Sequence Diagram illustrates the chronological interaction between the user, dashboard, repository analysis module, AI processing module, execution engine, and database components.

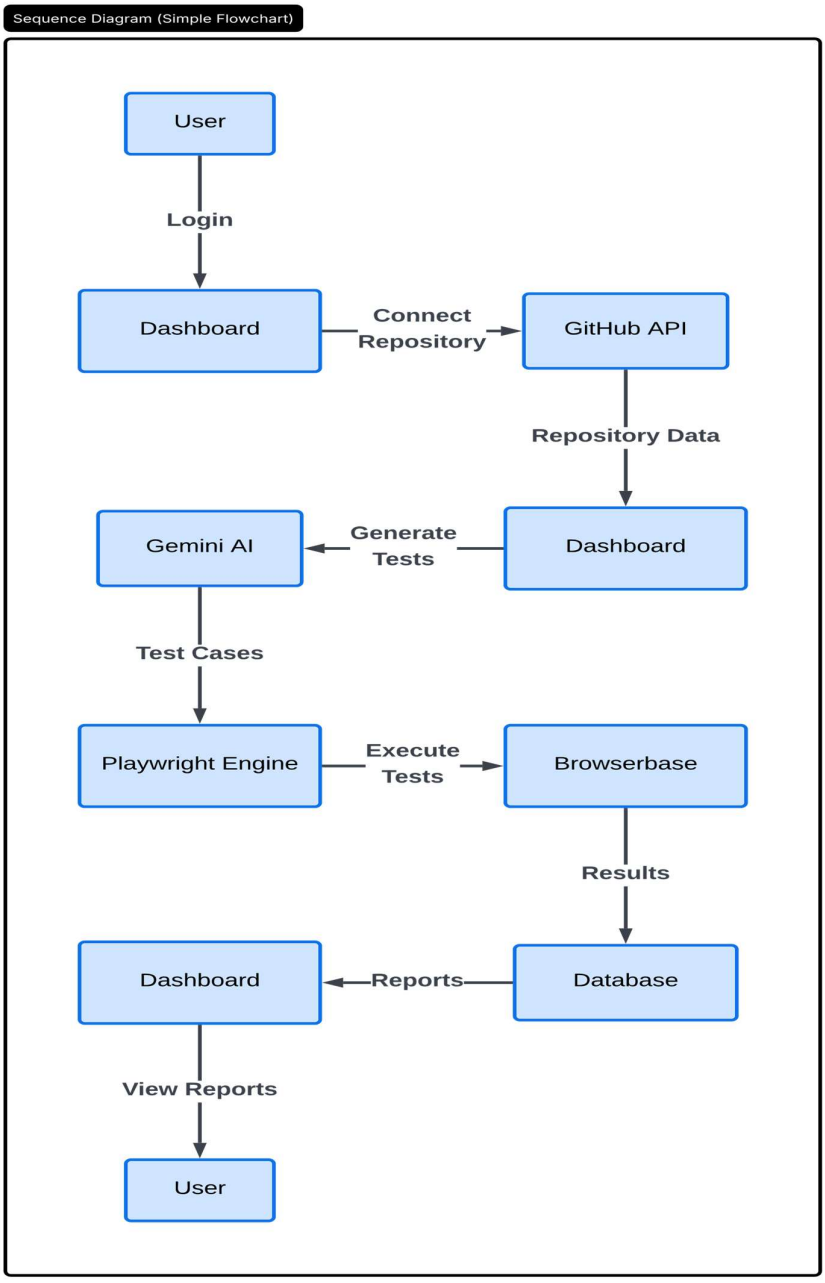


Figure 3.6 – Sequence Diagram

3.4.4 Activity Diagram

The Activity Diagram represents the operational workflow followed by the AI SYSTEM FOR AUTONOMOUS TESTING from user authentication to report generation.

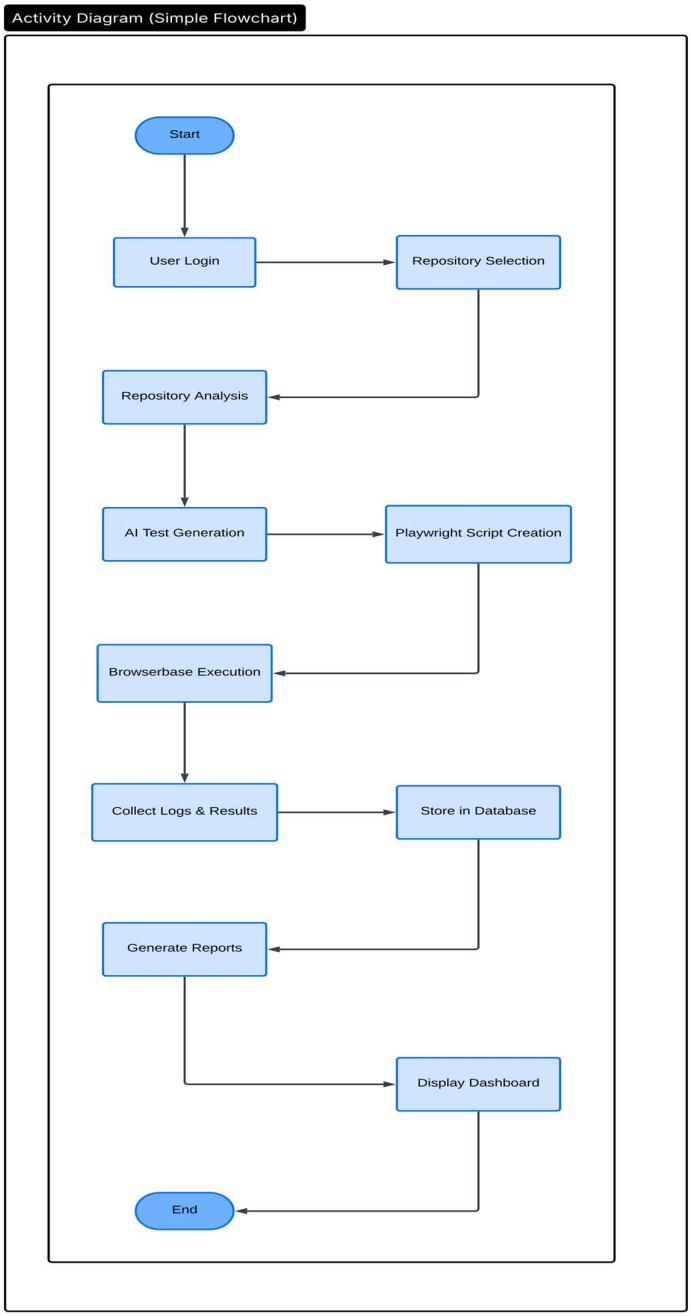


Figure 3.7 – Activity Diagram

CHAPTER 4 – IMPLEMENTATION

4.1 Technologies and Tools Used

The implementation of the Autonomous AI Software Quality Assurance Platform was carried out using a modern cloud-native technology stack. The selected technologies were chosen based on scalability, security, maintainability, and their ability to support artificial intelligence driven automation workflows.

The platform follows a full-stack architecture where the frontend, backend, database, artificial intelligence services, browser automation layer, and cloud execution infrastructure work together to provide autonomous software testing capabilities.

The major technologies used in the project are discussed below.

4.1.1 Next.js Framework

Next.js serves as the primary full-stack framework used for application development. It provides frontend rendering, server-side processing, API route management, routing mechanisms, and integration with external services. The framework acts as the central orchestrator of the entire platform.

4.1.2 React

React is utilized for building reusable user interface components. It enables dynamic rendering of repository information, generated test cases, execution logs, dashboards, and reporting modules. React Hooks are used extensively for state management and user interaction handling.

4.1.3 Tailwind CSS and Shadcn UI

Tailwind CSS provides utility-based styling while Shadcn UI offers pre-built accessible user interface components such as dialogs, buttons, accordions, tables, and forms. Together they ensure responsive and consistent user experiences across devices.

4.1.4 Clerk Authentication

Clerk provides secure user authentication, session management, and social sign-in capabilities. It handles identity verification while protecting internal application routes and maintaining user sessions securely.

4.1.5 GitHub OAuth and GitHub API

GitHub OAuth enables secure authorization to access user repositories. The GitHub API is used for fetching repository metadata, file trees, and source code content required for AI analysis and test generation.

4.1.6 Google Gemini AI

Google Gemini AI serves as the cognitive engine of the platform. It analyzes repository structures, generates intelligent test cases, creates Playwright automation scripts, and assists in autonomous quality assurance processes.

4.1.7 Playwright Core

Playwright Core is used as the browser automation framework responsible for navigating web applications, interacting with UI elements, and validating expected application behavior. Unlike traditional automation frameworks, Playwright scripts are dynamically generated by Gemini AI.

4.1.8 Browserbase

Browserbase provides isolated cloud browser environments where AI-generated Playwright scripts are executed. It eliminates the need for local browser drivers and provides session recordings, execution logs, and browser telemetry.

4.1.9 Neon PostgreSQL

Neon Serverless PostgreSQL acts as the persistent data storage layer. It stores users, repositories, configurations, generated test cases, execution logs, reports, and billing information.

4.1.10 Drizzle ORM

Drizzle ORM provides type-safe communication between the application backend and PostgreSQL database. It simplifies schema management while preventing SQL injection vulnerabilities through parameterized queries.

4.1.11 Stripe

Stripe is integrated for billing management and credit tracking. It monitors AI usage, repository additions, test generation operations, and execution activities within the SaaS platform.

4.2 Module Description

The Autonomous AI QA Platform is divided into multiple functional modules. Each module performs a dedicated responsibility while collaborating with other modules through secure APIs and service integrations.

The modular architecture improves maintainability, scalability, and future extensibility of the platform.

4.2.1 User Authentication Module

This module manages user registration, login, session creation, and route protection. Clerk authentication services validate user identity while GitHub OAuth grants repository access permissions. Authenticated users can securely access only their own repositories and testing resources.

4.2.2 Repository Integration Module

The repository integration module connects with GitHub repositories through OAuth authorization. It retrieves repository metadata, directory structures, and source code files required for AI-driven analysis. The module filters irrelevant files such as node_modules and environment files before processing.

4.2.3 AI Test Generation Module

This module prepares repository context and sends structured prompts to Google Gemini AI. The AI analyzes application architecture, routes, business logic, and UI components to generate categorized test cases including UI, API, Authentication, and Integration tests.

4.2.4 Test Management Module

Generated test cases are stored within the database and presented through the user interface. Users can review, modify, prioritize, and selectively execute generated tests before actual execution begins.

4.2.5 Script Generation Module

When execution is requested, the system submits test-specific information back to Gemini AI. The model generates executable Playwright scripts customized for the target application's workflow and routing structure.

4.2.6 Execution Module

Generated Playwright scripts are transmitted to Browserbase. Browserbase provisions isolated Chromium browser instances and executes scripts through the Chrome DevTools Protocol (CDP). This ensures secure cloud-based execution.

4.2.7 Reporting Module

Execution results, logs, screenshots, session URLs, and pass/fail statuses are collected and stored within the database. The reporting module presents these results through dashboards, execution summaries, and visual debugging interfaces.

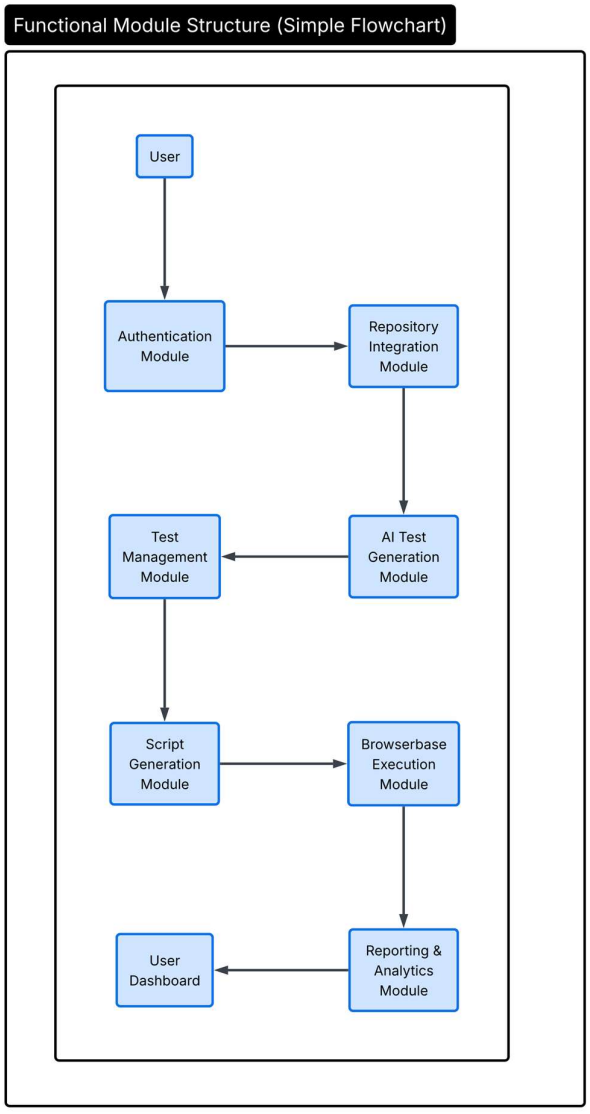


Figure 4.1 – Functional Module Structure

4.3 Algorithm / Methodology

The proposed system follows an autonomous testing methodology that integrates repository analysis, Artificial Intelligence, browser automation, and cloud execution. Unlike traditional frameworks requiring manual scripting, it automatically generates and executes tests based on application context. The workflow includes repository analysis, AI-driven test generation, script creation, cloud execution, and result reporting.

Algorithm: Autonomous Test Generation and Execution

Step 1:

Authenticate user using Clerk.

Step 2:

Authorize GitHub repository access.

Step 3:

Fetch repository tree from GitHub API.

Step 4:

Filter unnecessary directories and files.

Step 5:

Extract source code context.

Step 6:

Prepare structured AI prompt.

Step 7:

Send repository context to Gemini AI.

Step 8:

Generate structured test cases.

Step 9:

Store generated test cases in database.

Step 10:

User selects tests for execution.

Step 11:

Generate Playwright automation script.

Step 12:

Submit script to Browserbase.

Step 13:

Execute script in cloud browser.

Step 14:

Capture execution logs and results.

Step 15:

Store execution telemetry.

Step 16:

Generate reports and analytics.

Step 17:

Display results to user.

End.

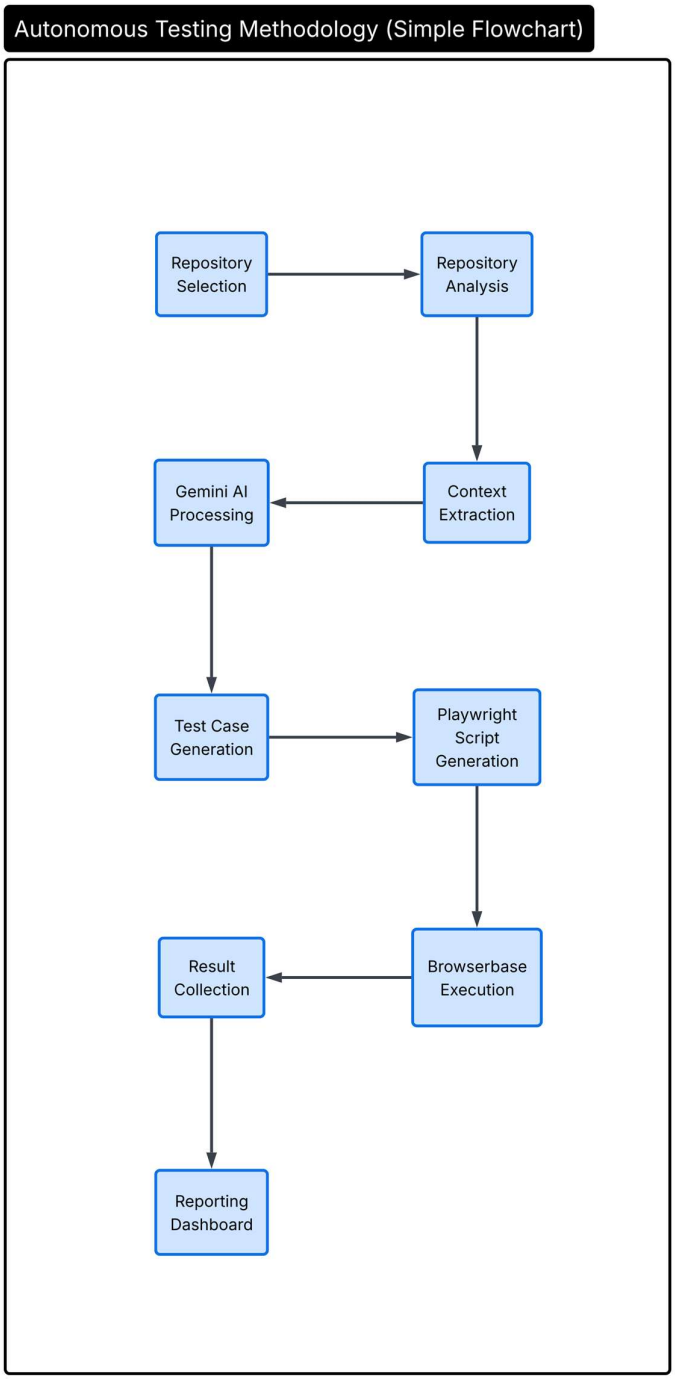


Figure 4.2 – Autonomous Testing Methodology

The methodology enables complete automation of software testing activities while minimizing manual intervention. Repository analysis, AI processing, script creation, execution, and reporting occur as a continuous pipeline.

4.4 Code Implementation

The implementation of the Autonomous AI QA Platform follows a layered architecture separating frontend presentation logic, backend business processing, artificial intelligence services, cloud execution infrastructure, and database persistence.

The source code is organized using the Next.js application directory structure, enabling modular development and efficient maintenance.

4.4.1 Authentication Implementation

User authentication is implemented using Clerk middleware and protected routes. GitHub OAuth authorization enables secure access to user repositories while OAuth tokens are stored using HTTP-only cookies to enhance security.

4.4.2 Repository Analysis Implementation

The backend retrieves repository file trees through GitHub API endpoints. Files are filtered according to supported extensions such as .js, .ts, and .tsx while excluding unnecessary directories. Repository context is then prepared for AI processing.

4.4.3 AI Integration Implementation

Google Gemini AI is integrated using the Google Gen AI SDK. Structured prompts are constructed using repository context, user instructions, and predefined response schemas. Generated outputs are validated before storage.

4.4.4 Playwright Script Generation

The platform dynamically generates Playwright automation scripts instead of relying on static test files. Generated scripts contain navigation actions, assertions, and validation steps tailored to the analyzed application.

4.4.5 Cloud Execution Implementation

Browserbase receives generated Playwright scripts and executes them within isolated browser sessions. Execution telemetry including logs, session IDs, recordings, and status information is returned to the backend.

4.4.6 Database Implementation

Database interactions are performed through Drizzle ORM. The system stores:

- User Information
- Repository Metadata
- Project Configurations
- Generated Test Cases
- Execution Logs
- Session URLs
- Billing Information

within Neon PostgreSQL databases.

4.4.7 Security Implementation

Security controls implemented within the system include:

- Clerk Authentication
- GitHub OAuth Authorization
- HTTP-Only Cookie Storage
- Route Protection Middleware
- Drizzle ORM Parameterized Queries
- Environment Variable Secret Management
- AI Response Schema Validation
- Browserbase Sandboxed Execution

These controls protect user information, repository contents, and cloud execution environments from unauthorized access and security threats.

CHAPTER 5 – TESTING AND RESULTS

5.1 Testing Strategy

The Autonomous AI Software Quality Assurance Platform was tested using functional, integration, execution, and security testing techniques to validate system reliability and performance. The primary objective was to verify that the platform could successfully analyze software repositories, generate intelligent test cases, execute browser automation scripts, and produce accurate reporting results. Testing was conducted across all major modules, including user authentication, repository integration, AI processing, test generation, browser automation, database operations, and reporting functionalities. The testing workflow involved user authentication, repository analysis, AI-driven test generation, Playwright script creation, Browserbase cloud execution, and result visualization to ensure seamless end-to-end operation of the platform.

5.1.1 Functional Testing

Functional testing was performed to verify that every module operated according to the specified requirements.

Module	Testing Objective	Result
Authentication Module	Verify user login and access control	Passed
GitHub Integration	Verify repository retrieval and authorization	Passed
Repository Analysis	Verify source code extraction and filtering	Passed
Test Generation	Verify AI-generated test case creation	Passed
Playwright Generation	Verify automation script generation	Passed
Browserbase Execution	Verify cloud execution workflow	Passed
Reporting Dashboard	Verify result visualization and logs	Passed

Table 5.1 Functional Testing Validation

The platform successfully completed all major functional requirements without critical failures.

5.1.2 Integration Testing

Integration testing was conducted to ensure proper communication between interconnected services.

The following integrations were verified:

- Clerk Authentication ↔ Next.js Application
- GitHub OAuth ↔ Repository Access Layer
- GitHub API ↔ Gemini AI
- Gemini AI ↔ Playwright Script Generation
- Browserbase ↔ Playwright Execution Engine
- Browserbase ↔ Neon Database
- Neon Database ↔ Dashboard Interface
- Stripe ↔ Credit Management System

The successful execution of end-to-end workflows demonstrated seamless communication among all integrated components.

5.1.3 End-to-End Workflow Testing

An end-to-end testing approach was employed to validate the complete autonomous workflow.

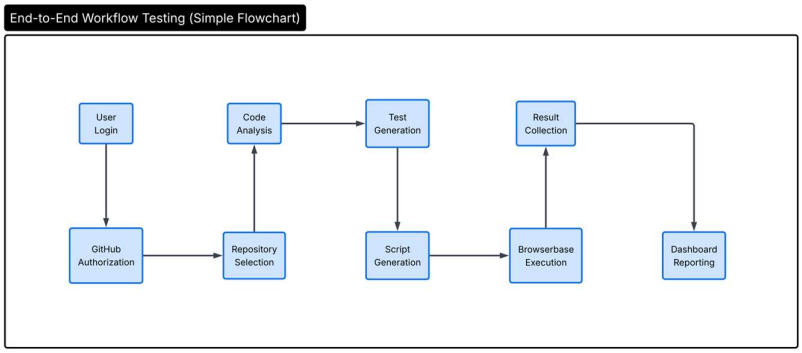


Figure 5.1 – End-to-End Workflow Testing Process

The entire workflow executed successfully, confirming the correctness of the implemented architecture.

5.2 Test Cases and Results

To evaluate the effectiveness of the proposed system, multiple test scenarios were executed across various web application components. The generated test cases included user interface validation, authentication verification, navigation testing, and integration testing.

The generated test cases were automatically created by Google Gemini AI after analyzing repository source code and application routing structures. These test cases were subsequently converted into executable Playwright automation scripts and executed within Browserbase cloud environments.

Test ID	Test Case	Category	Expected Result
TC-01	Verify User Login	Authentication	User successfully logs in
TC-02	Verify Dashboard Loading	UI Testing	Dashboard loads correctly
TC-03	Verify Repository Selection	Functional Testing	Repository added successfully
TC-04	Verify Test Case Generation	AI Validation	Test cases generated
TC-05	Verify Browser Execution	Integration Testing	Browser session executes successfully

Table 5.2 Sample Test Cases Generated by the System

The generated test cases followed a structured format containing:

- Test Title
- Description
- Priority
- Test Type
- Expected Result
- Target Route
- Target Files

which were generated dynamically using repository context and AI analysis.

Execution Results

The generated Playwright scripts were executed through Browserbase cloud sessions.

Test Case	Status	Execution Result
Login Validation	Pass	User authenticated successfully
Dashboard Rendering	Pass	Dashboard displayed correctly
Repository Integration	Pass	Repository connected successfully
Test Generation	Pass	Test cases generated correctly
Browser Execution	Pass	Browser automation completed

Table 5.3 Sample Execution Summary.

The Browserbase execution environment successfully returned:

- Pass/Fail Status
- Session Identifier
- Execution Logs
- Video Recording URL
- Browser Telemetry Data

These results were stored within Neon PostgreSQL and displayed on the reporting dashboard.

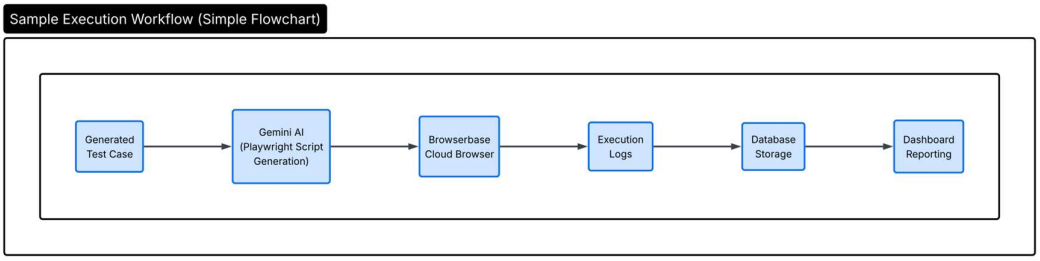


Figure 5.2 - Sample Execution Workflow

5.3 Performance Analysis

Performance analysis was conducted to evaluate the effectiveness, reliability, scalability, and security of the Autonomous AI QA Platform.

The evaluation focused on repository processing efficiency, AI test generation accuracy, cloud execution performance, execution reporting, security mechanisms, and overall system limitations.

5.3.1 Repository Analysis Performance

The repository analysis engine successfully processed source code directly from GitHub repositories without requiring local cloning.

Key observations include:

- Efficient filtering of irrelevant directories.
- Reduced processing overhead.
- Optimized token consumption.
- Faster repository context generation.

The selective extraction of .js, .ts, and .tsx files improved AI processing efficiency and reduced unnecessary computational costs.

5.3.2 AI Test Generation Performance

Google Gemini AI successfully generated structured and categorized test cases based on repository architecture and application logic.

Performance observations:

- Automated identification of test scenarios.
- Structured JSON-based outputs.
- Reduced manual test design effort.
- Faster generation compared to manual QA workflows.

The generated test cases covered UI, Authentication, Integration, and Functional Testing categories.

5.3.3 Browser Automation Performance

The Browserbase and Playwright integration demonstrated reliable cloud-based execution capabilities.

Performance highlights:

- No local browser installation required.
- Consistent execution environments.
- Automated session recording.
- Detailed execution telemetry collection.
- Efficient failure diagnostics.

The use of Browserbase significantly reduced environment-related testing issues commonly encountered in traditional automation frameworks.

5.3.4 Security Analysis

The platform incorporates multiple layers of security to protect user data, repositories, API credentials, and execution environments.

Authentication Security

- Clerk-based authentication
- Session validation
- Protected application routes

Authorization Security

- GitHub OAuth authorization
- User-specific resource isolation
- Access-controlled repository management

API Security

- Server-side identity verification
- Secure webhook validation

- Protected backend endpoints

Secret Management

- Environment variable storage
- Restricted server-side processing
- API key isolation

Execution Security

- Browserbase sandboxing
- Ephemeral cloud browser sessions
- Isolation of AI-generated code

These security controls significantly reduce risks associated with unauthorized access, credential leakage, remote code execution attacks, and data exposure.

5.3.5 Advantages of the Proposed System

The implemented platform offers several significant advantages over traditional software testing approaches.

Automated Test Creation: The system automatically generates test cases without requiring manual scripting.

Reduced Testing Time: AI-assisted generation and execution significantly reduce testing effort.

Cloud-Based Execution: Browserbase removes dependency on local browser drivers and testing environments.

Improved Scalability: The cloud-native architecture enables parallel execution and easier scaling.

Enhanced Debugging: Session recordings and detailed execution logs simplify issue diagnosis.

Modern SaaS Architecture: The platform combines artificial intelligence, browser automation, and cloud services within a unified environment.

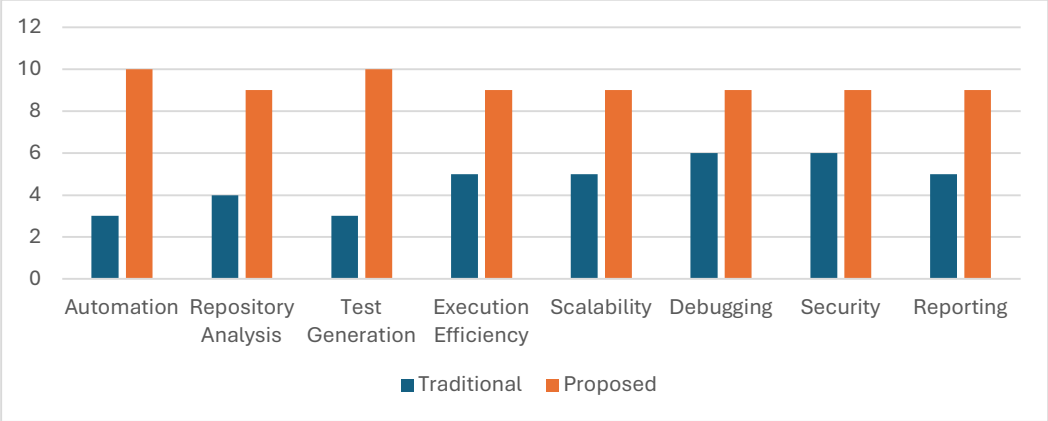


Figure 5.1 – Performance Comparison Graph

Figure 5.1 shows the comparative performance of traditional software testing and the proposed AI SYSTEM FOR AUTONOMOUS TESTING. The proposed system demonstrates better automation, efficiency, scalability, and reporting capabilities.

Performance Metric	**Traditional Testing Approach**	**AI SYSTEM FOR AUTONOMOUS TESTING**
Test Case Creation Time	High (Manual Effort)	Low (AI-Generated)
Repository Analysis Speed	Manual Code Review	Automated Repository Scanning
Test Script Development	Manual Coding Required	Automatically Generated
Execution Environment Setup	Local Configuration Required	Cloud-Based Browser Execution
Browser Compatibility Testing	Limited	Scalable Cloud Execution
Test Execution Consistency	Environment Dependent	Standardized Execution Environment
Debugging Support	Manual Log Analysis	Automated Logs and Session Recordings
Scalability	Moderate	High
Resource Utilization	High Human Effort	Optimized Automation Workflow
Overall Testing Efficiency	Moderate	High

Table 5.2 – Performance Metrics Comparison

5.3.6 Limitations of the Proposed System

Despite its advantages, several limitations were observed.

AI Context Window Constraints: Very large repositories may exceed model context limitations.

Dynamic DOM Structures: Applications using highly dynamic CSS selectors may generate inaccurate locators.

Execution Latency: Cloud browser initialization introduces minor startup delays.

Dependency on External Services: The platform relies on GitHub, Gemini AI, Browserbase, and cloud infrastructure availability.

AI Output Variability: Although schema validation improves reliability, AI-generated outputs may occasionally require human review.

Overall Evaluation

The testing results demonstrate that the Autonomous AI Software Quality Assurance Platform successfully achieves its intended objectives. The system effectively analyzes repositories, generates intelligent test cases, creates Playwright automation scripts, executes them within secure Browserbase environments, and provides detailed reporting capabilities.

The combination of artificial intelligence, cloud browser automation, secure execution environments, and real-time reporting establishes the platform as a practical and scalable solution for modern software quality assurance workflows.

CHAPTER 6 – CONCLUSION AND FUTURE WORK

6.1 Conclusion

The rapid growth of modern software applications has significantly increased the complexity of software testing processes. Traditional testing approaches often require extensive manual effort for designing test cases, maintaining automation scripts, configuring execution environments, and analyzing results. These challenges become more prominent in agile development environments where applications are continuously updated and deployed.

The AI SYSTEM FOR AUTONOMOUS TESTING was developed to address these challenges by introducing an intelligent testing platform capable of automating multiple stages of the software testing lifecycle. The system combines repository analysis, artificial intelligence, browser automation, cloud-based execution, and centralized reporting within a unified environment. By integrating these technologies, the platform reduces the dependency on manually authored automation scripts and simplifies the execution of software quality assurance activities.

The implemented solution successfully demonstrates the practical application of Artificial Intelligence in software testing. Through GitHub repository integration, the system is capable of analyzing project structures and extracting relevant contextual information. Google Gemini AI utilizes this context to generate structured test cases and create executable Playwright automation scripts. These scripts are executed securely within Browserbase cloud environments, allowing tests to run without requiring local browser installations or complex infrastructure configurations.

The platform further enhances software testing workflows through centralized management of repositories, generated test cases, execution logs, session recordings, and reporting dashboards. The use of Neon PostgreSQL and Drizzle ORM ensures efficient data management, while Clerk authentication provides secure access control and user management capabilities.

Testing and evaluation of the system confirmed the successful operation of all major modules including authentication, repository integration, AI-assisted test generation, Playwright script creation, cloud execution, and reporting functionalities. The results

demonstrate that the platform can effectively automate repetitive testing activities while providing meaningful execution feedback to developers and testers.

From an engineering perspective, the project successfully integrates multiple contemporary technologies including Artificial Intelligence, cloud computing, browser automation, serverless databases, and modern web development frameworks. The completed system serves as a practical example of how intelligent software agents can assist software quality assurance processes and contribute toward more efficient software development practices.

In conclusion, the AI SYSTEM FOR AUTONOMOUS TESTING successfully achieves its primary objectives by providing an automated, scalable, and intelligent testing environment capable of supporting modern web application testing requirements. The project establishes a strong foundation for future advancements in autonomous software quality assurance systems and demonstrates the growing potential of AI-assisted testing technologies within the Software Development Life Cycle.

6.2 Future Enhancements

Although the AI SYSTEM FOR AUTONOMOUS TESTING successfully achieves its intended objectives, several opportunities exist for further enhancement and expansion. Future improvements can increase automation capabilities, improve testing accuracy, and broaden the range of supported applications.

6.2.1 Self-Healing Automation

One of the most promising enhancements involves the implementation of self-healing automation mechanisms. When a generated Playwright script fails due to changes in user interface elements or application workflows, the system could automatically re-analyze the updated application structure and regenerate the affected automation logic without requiring human intervention.

6.2.2 Visual Regression Testing

The current implementation focuses primarily on functional validation. Future versions may incorporate computer vision and image comparison techniques to identify visual

inconsistencies, layout shifts, styling issues, and user interface regressions that are difficult to detect through traditional DOM-based testing methods.

6.2.3 Multi-Agent Collaboration

Future research may explore the development of specialized AI agents responsible for repository analysis, test generation, execution planning, debugging, and result interpretation. Such a multi-agent architecture could improve scalability and decision-making capabilities within autonomous testing environments.

6.2.4 Mobile Application Testing

The present implementation targets web applications developed using modern JavaScript frameworks. Future enhancements may extend support to Android and iOS applications through integration with frameworks such as Appium, enabling autonomous testing across mobile platforms.

6.2.5 Continuous Integration and Continuous Deployment Integration

The platform can be integrated directly with CI/CD pipelines to support automated testing during software build and deployment processes. This enhancement would enable organizations to perform intelligent testing automatically whenever source code changes are committed to repositories.

6.2.6 Security-Oriented Test Generation

Future versions may include advanced security testing capabilities such as authentication validation, authorization verification, API security testing, vulnerability detection, session management analysis, and automated security compliance assessments.

6.2.7 Adaptive Test Prioritization

Machine learning algorithms can be utilized to prioritize test cases based on historical execution outcomes, defect trends, code modification frequency, and application risk levels. This would improve testing efficiency by focusing resources on the most critical areas of an application.

6.2.8 Support for Large-Scale Enterprise Repositories

Current AI models may face challenges when processing extremely large repositories due to context limitations. Future implementations can incorporate repository segmentation, vector databases, retrieval-augmented generation techniques, and context optimization mechanisms to support enterprise-scale projects more effectively.

6.2.9 Advanced Analytics and Predictive Insights

The reporting module can be expanded to provide predictive quality metrics, defect probability analysis, execution trend visualization, and project health indicators. Such capabilities would assist project teams in making data-driven testing decisions.

6.2.10 Autonomous Feedback Loops

A future autonomous feedback mechanism could automatically analyze failed test executions, identify root causes, regenerate improved automation scripts, and re-execute tests without manual involvement. This capability would represent a significant step toward fully autonomous software quality assurance systems.

The implementation of these enhancements would transform the current platform into a more intelligent, adaptive, and comprehensive testing ecosystem capable of supporting the evolving requirements of modern software development environments.

REFERENCES

[1] A. Bertolino, “Software Testing Research: Achievements, Challenges, Dreams,” Future of Software Engineering (FOSE), IEEE Computer Society, pp. 85–103, 2007. [Accessed: May 24, 2026].

[2] M. Harman, Y. Jia, and Y. Zhang, “Achievements, Open Problems and Challenges for Search Based Software Testing,” IEEE 8th International Conference on Software Testing, Verification and Validation (ICST), pp. 1–12, 2015. [Accessed: Jun. 05, 2026].

[3] M. Felderer and W. Hasselbring, “Software Testing in the Age of Artificial Intelligence,” IEEE Software, vol. 38, no. 2, pp. 30–37, Mar.–Apr. 2021. [Accessed: May 14, 2026].

[4] S. E. et al., “Cloud-Based Software Testing: A Systematic Mapping Study,” IEEE Access, vol. 9, pp. 112042–112065, 2021. [Accessed: Jun. 01, 2026].

[5] M. Stocco, R. Yandrapally, and A. Mesbah, “Visual Web Test Breakage: Causes, Symptoms, and Remedies,” in Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering (ICSE), 2021, pp. 1171–1182. [Accessed: May 25, 2026].

[6] Z. Dong et al., “Self-Healing Test Scripts for Web Applications,” in Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE), 2022. [Accessed: Jun. 02, 2026].

[7] H. Pearce, B. Ahmad, B. Rieger, and R. Karri, “Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions,” in Proceedings of the IEEE Symposium on Security and Privacy (S&P), 2022, pp. 754–768. [Accessed: Jun. 04, 2026].

[8] J. E. Castro, A. Macedo, A. Silva, and R. Abreu, "A Comprehensive Empirical Study on Web Testing with Playwright, Cypress, and Selenium," Journal of Systems and Software, vol. 204, p. 111756, 2023. [Accessed: Jun. 03, 2026].

[9] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, “CODAMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models,” in Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE), 2023. [Accessed: May 29, 2026].

[10] F. Zhang et al., “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation,” in Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023. [Accessed: May 16, 2026].

[11] M. Schäfer, S. Nadi, A. Alshahwan, and F. Tip, “An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation,” IEEE Transactions on Software Engineering, vol. 50, no. 1, pp. 85–105, Jan. 2024. [Accessed: Jun. 07, 2026].

[12] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang, "Software Testing with Large Language Models: Survey, Landscape, and Vision," IEEE Transactions on Software Engineering, vol. 50, no. 4, pp. 886–905, 2024. [Accessed: May 18, 2026].

[15] Google, "Gemini API Documentation," Google AI for Developers, 2026. [Online]. Available: https://ai.google.dev/docs. [Accessed: May 14, 2026].

[16] Microsoft, "Playwright Documentation," Microsoft Corporation, 2026. [Online]. Available: https://playwright.dev/docs/intro. [Accessed: May 22, 2026].

[17] GitHub Inc., "GitHub REST API Documentation," GitHub Developer Documentation, 2026. [Online]. Available: https://docs.github.com/en/rest. [Accessed: Jun. 02, 2026].

[18] Browserbase, "Browserbase Documentation," Browserbase Developer Platform, 2026. [Online]. Available: https://docs.browserbase.com. [Accessed: May 11, 2026].

[19] Neon Inc., "Neon Serverless PostgreSQL Documentation," Neon Database Documentation, 2026. [Online]. Available: https://neon.tech/docs/introduction. [Accessed: Jun. 07, 2026].

[20] Drizzle Team, "Drizzle ORM Documentation," Drizzle ORM Official Documentation, 2026. [Online]. Available: https://orm.drizzle.team/docs/overview. [Accessed: May 28, 2026].

[21] Clerk Inc., "Authentication and User Management Documentation," Clerk Developer Documentation, 2026. [Online]. Available: https://clerk.com/docs. [Accessed: May 19, 2026].

[22] Next.js Team, "Next.js Official Documentation," Vercel, 2026. [Online]. Available: https://nextjs.org/docs. [Accessed: Jun. 04, 2026].

APPENDIX A – SOURCE CODE

A.1 Authentication Module

Purpose

Handles user authentication, protected route access, and session validation using Clerk middleware.

Representative Code Snippet (from proxy.ts)

```
import { clerkMiddleware, createRouteMatcher } from '@clerk/nextjs/server'


const isProtectedRoute = createRouteMatcher(['/workspace(.*)'])


export default clerkMiddleware(async (auth, req) => {
  const searchParams =
    (req as any).nextUrl?.searchParams ??
    new URL(req.url).searchParams


  if (searchParams.has('__clerk_handshake')) return


  if (isProtectedRoute(req)) await auth.protect()
})
```

Explanation

The middleware protects workspace routes and ensures that only authenticated users can access testing resources. Unauthorized users are redirected through Clerk's authentication flow.

A.2 GitHub Integration Module

Purpose

Provides GitHub OAuth authentication and repository synchronization.

Representative Code Snippet (from github/callback/route.ts)

```
const res = await fetch(
  "https://github.com/login/oauth/access_token",
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Accept: "application/json",
    },
    body: JSON.stringify({
      client_id: process.env.GITHUB_CLIENT_ID,
      client_secret: process.env.GITHUB_CLIENT_SECRET,
      code,
    }),
  }
);
response.cookies.set("gh_token", token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === "production",
  sameSite: "lax",
});
```

Explanation

The module exchanges the GitHub authorization code for an access token and stores it securely using HTTP-only cookies.

A.3 AI Test Generation Module

Purpose

Analyzes repository files and generates structured test cases using Google Gemini AI.

Representative Code Snippet (from generate-test-cases/route.ts)

```
const ALLOWED_EXTENSIONS = new Set([
  ".js",
  ".jsx",
  ".ts",
  ".tsx",
  ".json",
  ".md",
]);

function isUsefulFile(path: string): boolean {
  const pathSegments = path.toLowerCase().split("/");

  const isIgnored = pathSegments.some(
    (segment) => IGNORE_PATHS.has(segment)
  );
  if (isIgnored) return false;
  return ALLOWED_EXTENSIONS.has(
    path.substring(path.lastIndexOf("."))
  );
}
```

Explanation

The system filters repository contents and sends only relevant files to Gemini AI, reducing token usage and improving test generation quality.

A.4 Playwright Script Generation Module

Purpose

Creates executable Playwright automation scripts dynamically through Gemini AI.

Representative Code Snippet

```
const ai = new GoogleGenAI({

    apiKey: process.env.GEMINI_API_KEY!,

});

const { testCaseId, baseUrl } = body;

const [testCase] = await db

    .select()

    .from(TestCasesTable)

    .where(eq(TestCasesTable.id, testCaseId));
```

Explanation

The module retrieves the selected test case, prepares context information, and requests Gemini AI to generate Playwright automation logic specific to the target application.

A.5 Browserbase Execution Module

Purpose

Executes generated Playwright scripts within isolated cloud browser environments.

Representative Code Snippet

```
const bb = new Browserbase({

    apiKey: process.env.BROWSERBASE_API_KEY!,

});

import { chromium } from "playwright-core";
```

Explanation

Browserbase provisions temporary Chromium instances and executes AI-generated Playwright scripts. Execution telemetry, logs, and recordings are collected automatically.

A.6 Database Layer

Purpose

Stores repositories, test cases, logs, session recordings, and user information.

Representative Code Snippet (from schema.ts)

```
export const users = pgTable("users", {
  id: serial("id").primaryKey(),
  name: text("name"),
  email: text("email").notNull().unique(),
  credits: integer("credits").default(1000).notNull(),
});
export const repositories = pgTable("repositories", {
  id: serial("id").primaryKey(),
  userId: integer("user_id").notNull(),
  name: text("name").notNull(),
});
export const TestCasesTable = pgTable("test_cases", {
  id: serial("id").primaryKey(),
  title: varchar("title", { length: 500 }).notNull(),
  status: varchar("status", { length: 100 }).default("generated"),
});
```

Explanation

The schema maintains relationships between users, repositories, generated test cases, execution logs, and Browserbase session information.

A.7 Dashboard and Reporting Module

Purpose

Displays testing metrics, execution results, logs, and session recordings.

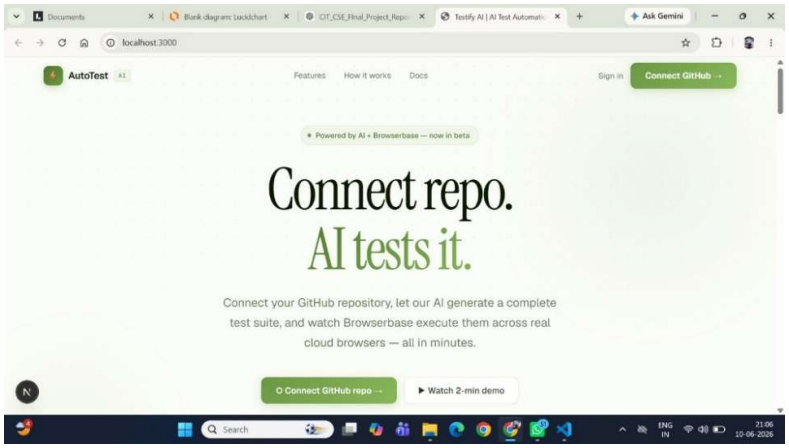
Representative Code Snippet (workspace/page.tsx)

```
function Workspace() {

    return (

        <div className='mx-auto max-w-4xl p-10'>

            <WorkspaceBody />

        </div>

    )

}
```

Explanation

The dashboard serves as the central interface where users manage repositories, generate tests, monitor executions, and review testing results.

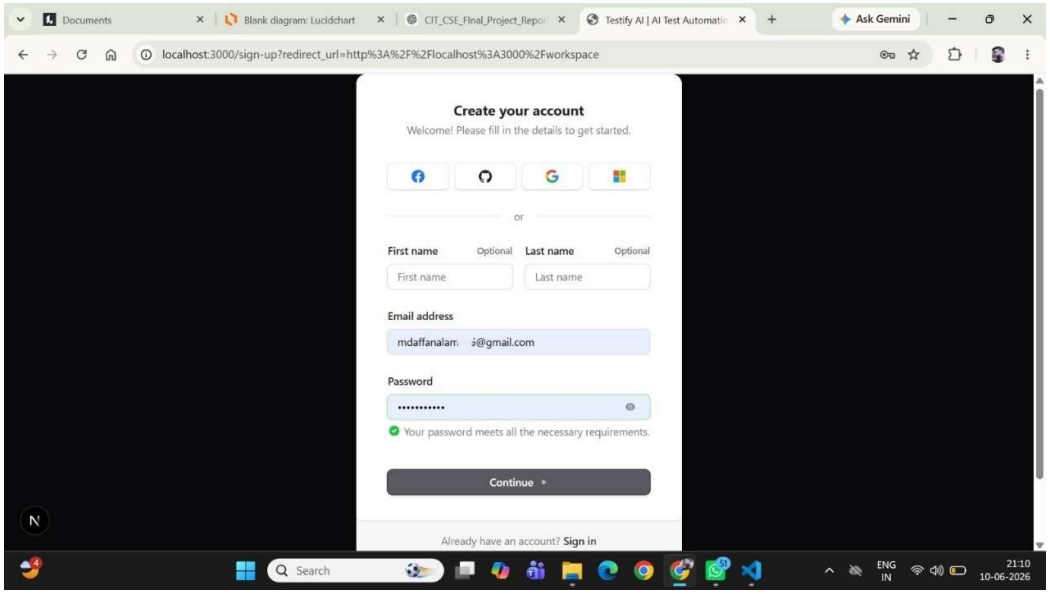
APPENDIX B – SCREENSHOTS



Screenshot B.1 – Landing Page

Description:

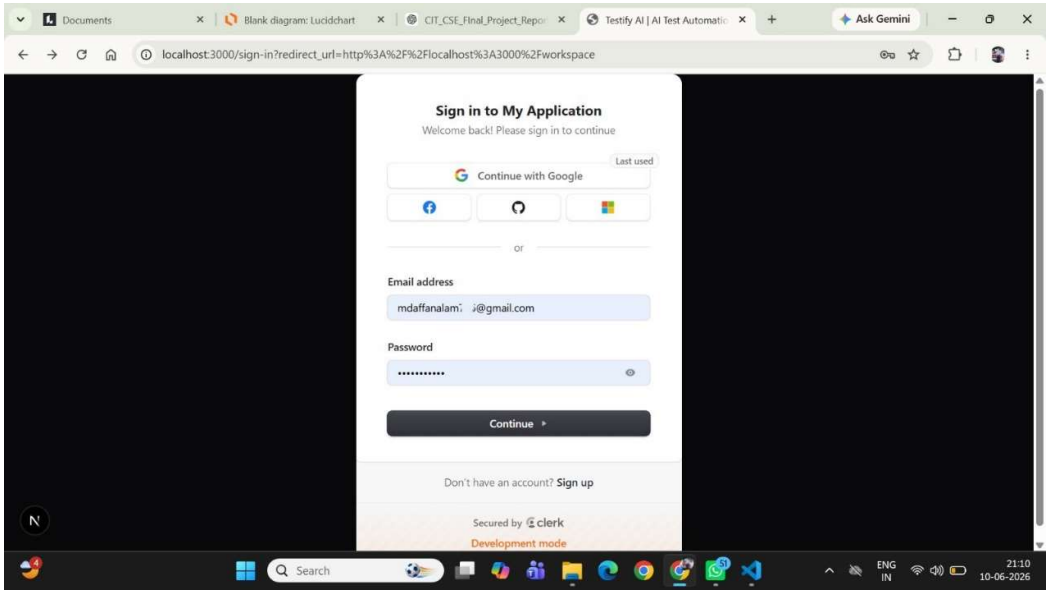
Landing page of the AutoTest AI platform. The page introduces the system, highlights AI-powered autonomous web testing capabilities, and provides GitHub repository integration options for automated test generation.



Screenshot B.2 – User Registration Interface

Description:

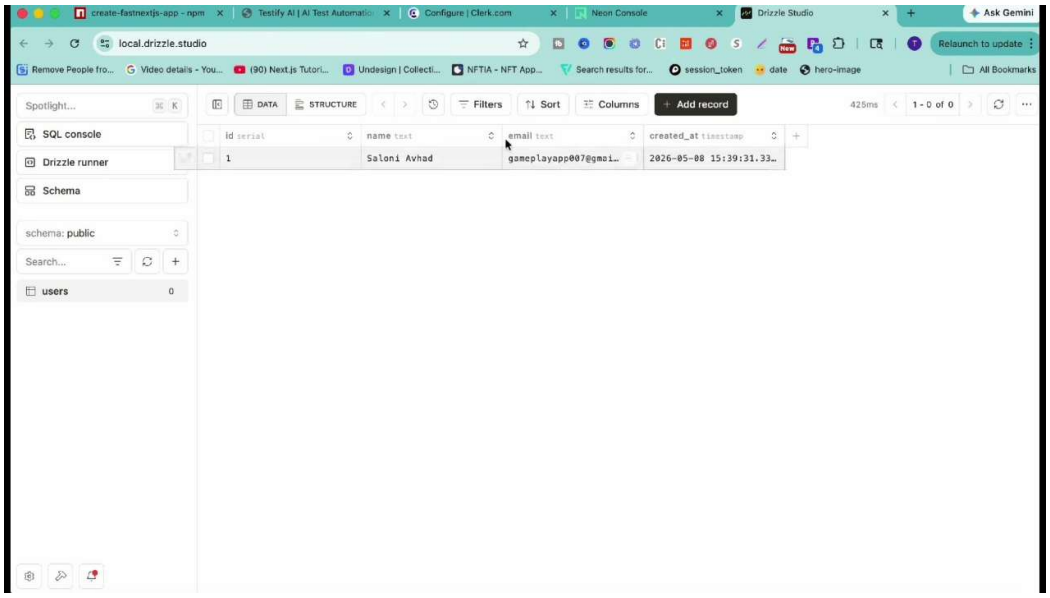
User registration page powered by Clerk Authentication. New users can create accounts using email credentials or social login providers before accessing the testing workspace.



Screenshot B.3 – User Login Interface

Description:

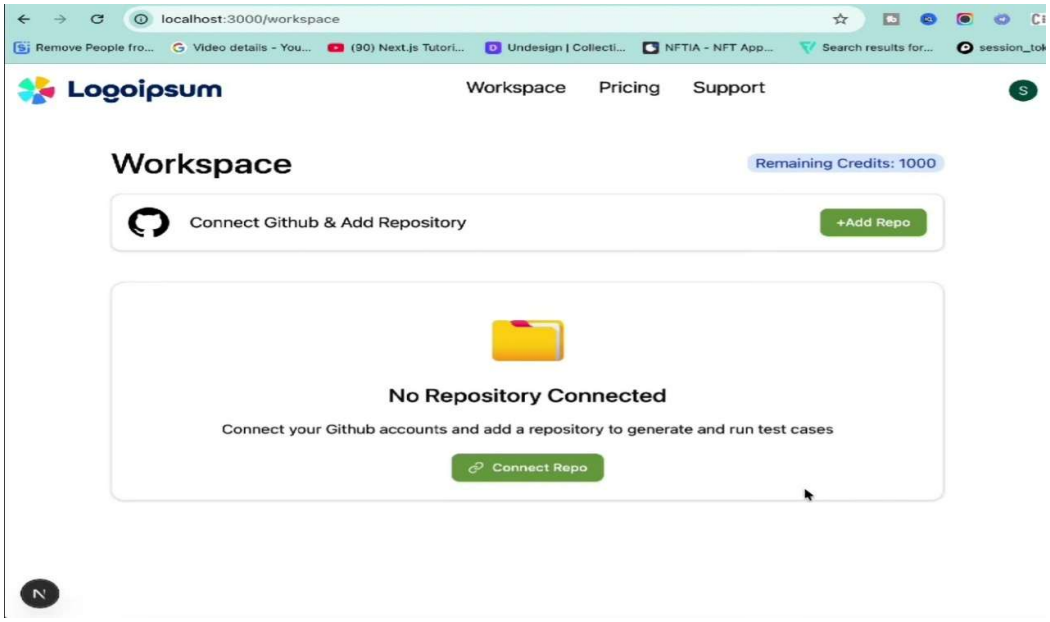
Secure login page implemented using Clerk Authentication. Registered users can authenticate using email credentials or third-party identity providers.



Screenshot B.4 – Database Management Dashboard

Description:

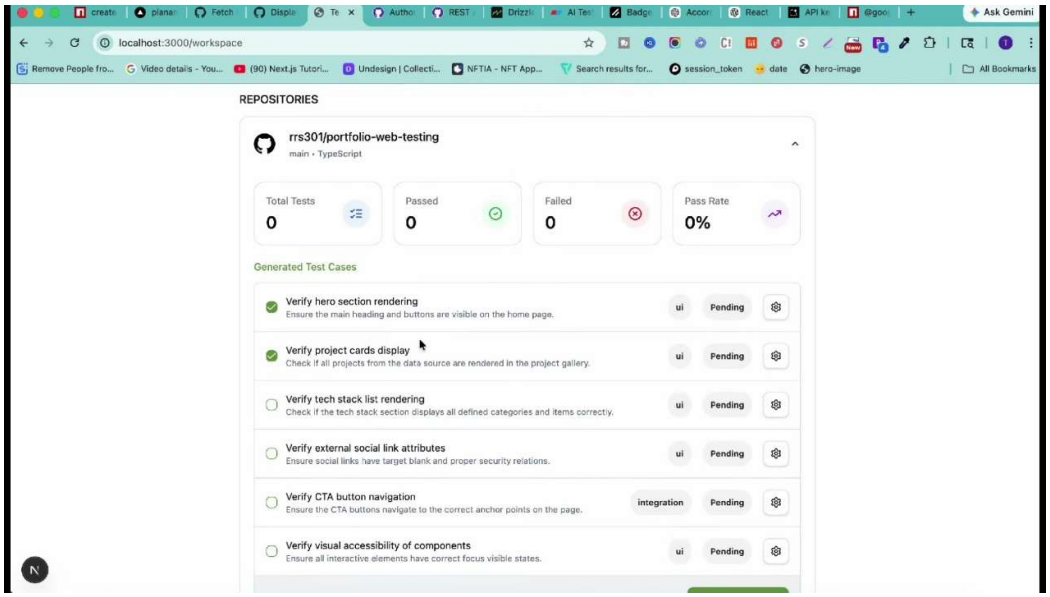
Drizzle Studio database management interface connected to Neon PostgreSQL. The screenshot displays stored user records and database schema used by the platform.



Screenshot B.5 – Workspace Before Repository Connection

Description:

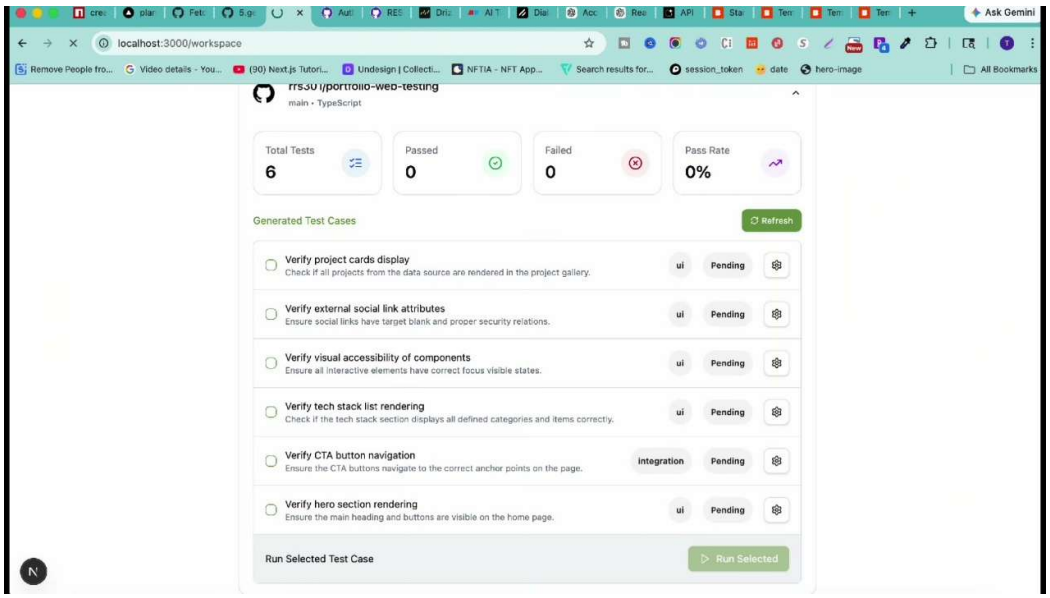
Initial workspace dashboard displayed after authentication. Users are prompted to connect a GitHub repository before generating AI-driven test cases.



Screenshot B.6 – Repository Connected Dashboard

Description:

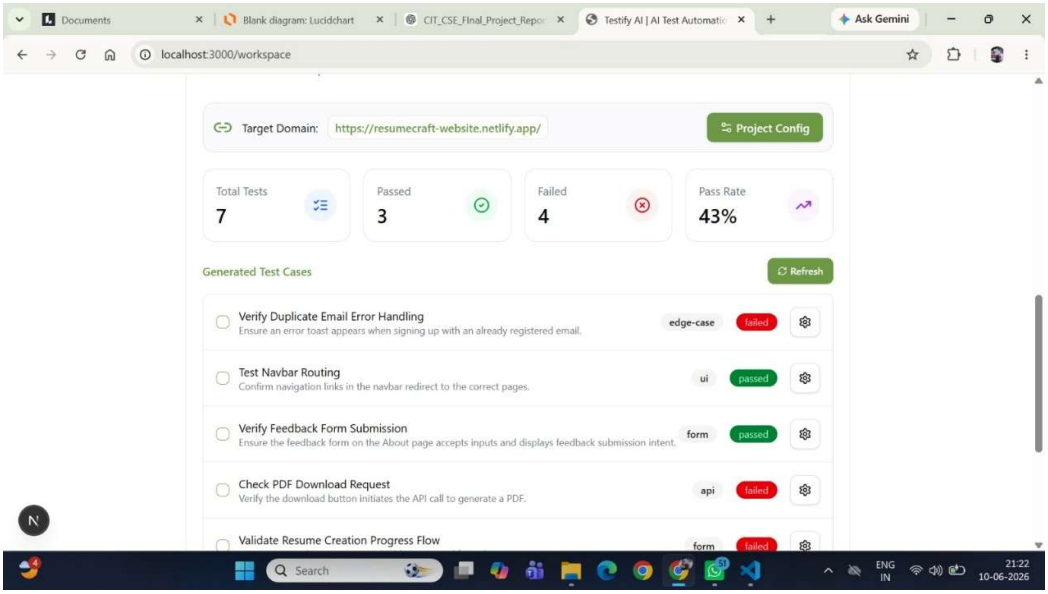
Workspace dashboard after successful GitHub repository integration. Repository information and testing metrics are displayed, enabling AI-based test generation.



Screenshot B.7 – AI Generated Test Cases

Description:

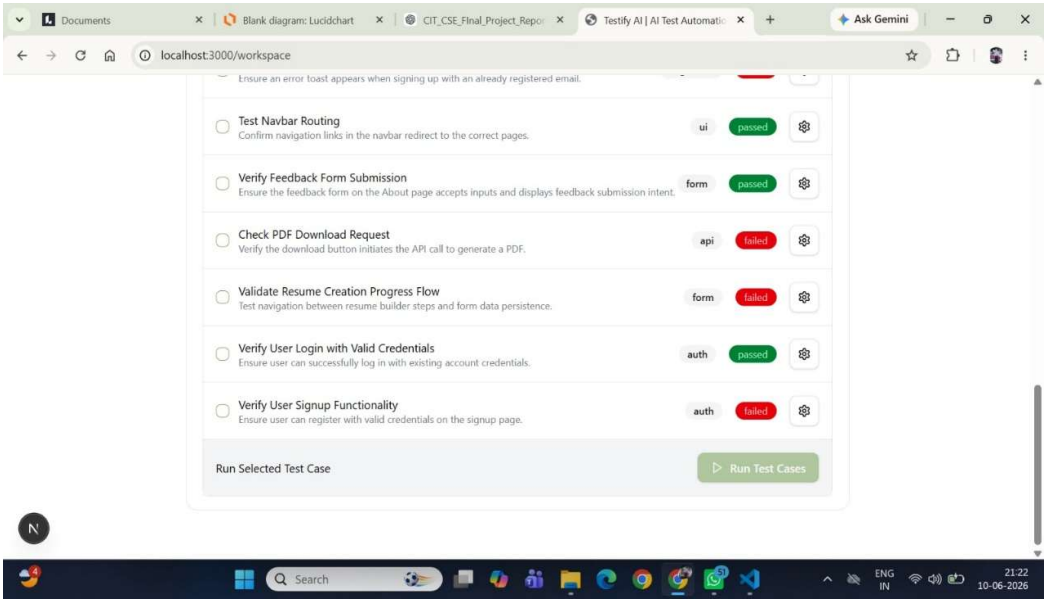
Output generated by the Test Generation Agent. The system automatically analyzes repository structure and produces categorized UI and integration test scenarios.



Screenshot B.8 – Automated Test Execution Results

Description:

Execution results produced after running generated test cases. The dashboard displays passed tests, failed tests, execution status, and overall pass-rate metrics.



Screenshot B.9 – Detailed Test Case Status View

Description:

Detailed test execution panel showing individual test cases, execution categories, authentication validation tests, API validation tests, and result statuses.